

**LAPORAN AKHIR PRAKTIKUM
ALGORITMA & STRUKTUR DATA**



Oleh:

Rahma Syarita Jaya

NIM. 2510817320004

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
JUNI
2025**

LEMBAR PENGESAHAN
LAPORAN AKHIR PRAKTIKUM ALGORITMA & STRUKTUR
DATA

Laporan Praktikum Algoritma & Struktur Data

Modul 1 : Struck and Pointer

Modul 2 : Stack and Queue

Modul 3 : Single Linked List Circular and Double Linked List Circular

Modul 4 : Sorting

Modul 5 : Searching

Modul 6 : Graph and Tree

ini disusun sebagai syarat lulus mata kuliah Praktikum Praktikum Algoritma & Struktur Data. Laporan Akhir Praktikum ini dikerjakan oleh:

Nama Praktikan : Rahma Syarita Jaya

NIM : 2510817320004

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Fauzan Ahsani
NIM. 2310817310009

Ir. Muhamamd Alkaff, S,Kom, M.Kom,
Ph.D NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN.....	ii
DAFTAR ISI	iii
DAFTAR GAMBAR.....	vii
DAFTAR TABEL	viii
MODUL 1: STRUCT AND POINTER	9
SOAL 1.....	9
A. Source Code	9
B. Output Program.....	10
C. Pembahasan.....	10
SOAL 2.....	12
A. Source Code	12
B. Output Program.....	13
C. Pembahasan.....	13
MODUL 2: STACK AND QUEUE	14
SOAL 1: Implementasi Stack.....	14
A. Source Code	15
B. Pembahasan.....	16
SOAL 2: Si Palui dan Mesin Hitung Aneh	17
A. Source Code	18
B. Output Program.....	19
C. Pembahasan.....	19
SOAL 3: Implementasi Queue	21
A. Source Code	21
B. Pembahasan.....	22
SOAL 4: Si Palui dan Deretan Angka.....	24
A. Source Code	25
B. Output Program.....	26
C. Pembahasan.....	26
MODUL 3: LINKED LIST	28
SOAL 1: Implementasi Senarai Berantai Tunggal Melingkar	28
A. Source Code	28

B. Pembahasan.....	32
SOAL 2: Si Palui dan Mesin Hitung Aneh	35
A. Source Code	36
B. Output Program.....	37
C. Pembahasan.....	37
SOAL 3: Implementasi Senarai Berantai Ganda Melingkar	39
A. Source Code	39
B. Pembahasan.....	42
SOAL 4: Si Palui dan Deretan Angka.....	45
A. Source Code	46
B. Output Program.....	47
C. Pembahasan.....	47
MODUL 4: SORTING	48
Bubble Sort.....	48
A. Source Code	48
B. Output Program.....	48
C. Pembahasan.....	49
Selection Sort.....	50
A. Source Code	50
B. Output Program.....	50
C. Pembahasan.....	51
Insertion Sort	52
A. Source Code	52
B. Output Program.....	52
C. Pembahasan.....	53
Merge Sort	54
A. Source Code	54
B. Output Program.....	55
C. Pembahasan.....	55
Quick Sort.....	56
A. Source Code	56
B. Output Program.....	57
C. Pembahasan.....	57
Radix Sort.....	59

A.	Source Code	59
B.	Output Program	60
C.	Pembahasan	60
	KESIMPULAN	62
	MODUL 5: SEARCHING	64
	LINEAR SEARCH	64
A.	Source Code	64
B.	Output Program	64
C.	Pembahasan	64
	BINARY SEARCH	66
A.	Source Code	66
B.	Output Program	66
C.	Pembahasan	67
	MODUL 6: GRAPH AND TREE	70
	SOAL 1: Konversi Adjacency List ke Adjacency Matrix	70
A.	Source Code	71
B.	Output Program	73
C.	Pembahasan	73
	SOAL 2: Konversi Adjacency Matrix ke Adjacency List	74
A.	Source Code	76
B.	Output Program	77
C.	Pembahasan	77
	Soal 3: BFS: Jarak Terpendek Keluar Labirin	78
A.	Source Code	79
B.	Output Program	81
C.	Pembahasan	81
	Soal 4: DFS: Menghitung Semua Jalur Keluar Labirin	83
A.	Source Code	86
B.	Output Program	87
C.	Pembahasan	87
	Soal 5: Tree Traversal	88
A.	Source Code	2
B.	Output Program	4
C.	Pembahasan	4

TAUTAN GIT	6
------------------	---

DAFTAR GAMBAR

Gambar 1. Screenshot Hasil Jawaban Soal 1 Modul 1	10
Gambar 2. Screenshot Hasil Jawaban Soal 2 Modul 1	13
Gambar 3. Screenshot 1 Hasil Jawaban Soal 1 Modul 2	19
Gambar 4. Screenshot 2 Hasil Jawaban Soal 1 Modul 2	19
Gambar 5. Screenshot 3 Hasil Jawaban Soal 1 Modul 2	19
Gambar 6. Screenshot 1 Hasil Jawaban Soal 2 Modul 2	26
Gambar 7. Screenshot 2 Hasil Jawaban Soal 2 Modul 2	26
Gambar 8. Screenshot 3 Hasil Jawaban Soal 2 Modul 2	26
Gambar 9. Screenshot Hasil Jawaban Soal 1 Modul 3	37
Gambar 10. Screenshot Hasil Jawaban Soal 2 Modul 3	47
Gambar 11. Output Algoritma Bubble Sort Modul 4	48
Gambar 13. Output Algoritma Selection Sort Modul 4	50
Gambar 14. Output Algoritma Insertion Sort Modul 4	52
Gambar 15. Output Algoritma Merge Sort Modul 4	55
Gambar 16. Output Algoritma Quick Sort Modul 4	57
Gambar 17. Output Algoritma Radix Sort Modul 4	60
Gambar 17. Output Algoritma Linear Search Modul 5	64
Gambar 18. Output Algoritma Binart Search Modul 5	66
Gambar 19. Screenshot Hasil Jawaban Soal 1 Modul 6	73
Gambar 20. Screenshot Hasil Jawaban Soal 2 Modul 6	77
Gambar 21. Screenshot Hasil Jawaban Soal 3 Modul 6	81
Gambar 22. Screenshot Hasil Jawaban Soal 4 Modul 6	87
Gambar 23. Screenshot Hasil Jawaban Soal 5 Modul 6	4

DAFTAR TABEL

Tabel 1. Source Code line.cpp Modul 1	9
Tabel 2. Source Code prob1.cpp Modul 1	10
Tabel 3. Source Code circle.cpp Modul 1	12
Tabel 4. Source Code prob2.cpp Modul 1	13
Tabel 5. Source code stack.cpp Modul 2.....	15
Tabel 6. Source code prob1.cpp Modul 2.....	18
Tabel 7. Source code queue.cpp Modul 2	21
Tabel 8. Source code prob2.cpp Modul 2.....	25
Tabel 9. Source code single_linked_list.cpp Modul 3	28
Tabel 10. Source code prob_a.cpp Modul 3	36
Tabel 11. Source code double_linked-list.cpp Modul 3	39
Tabel 12. Source code prob_b.cpp Modul 3.....	46
Tabel 13. Source code Bubble Sort Modul 4	48
Tabel 14. Source code Selection Sort Modul 4	50
Tabel 15. Source code Insertion Sort Modul 4.....	52
Tabel 16. Source code Merge Sort Modul 4.....	54
Tabel 17. Source code Quick Sort Modul 4	56
Tabel 18 Source code Radix Sort Modul 4	59
Tabel 19. Source code Linear Search Modul 5	64
Tabel 20. Source code Binary Search Modul 5	66
Tabel 21. Source code prob1.cpp Modul 6.....	71
Tabel 22 Source code prob2.cpp Modul 6.....	76
Tabel 23. Source code prob3.cpp Modul 6.....	79
Tabel 24. Source code prob4.cpp Modul 6.....	86
Tabel 25. Source code prob5.cpp Modul 6.....	2

MODUL 1: STRUCT AND POINTER

SOAL 1

Diberikan sebuah garis l yang direpresentasikan dalam bentuk persamaan umum:

$$ax + by + c = 0$$

dan sebuah titik $P = (x_P, y_P)$. Tentukan posisi titik P terhadap garis l dengan cara mengimplementasikan fungsi `gradient` dan `CheckPointPosition` pada file header yang telah diberikan.

Input	Output
1 0 -3 5 0	Left
1 0 -3 1 7	Right
2 -1 -1 1 1	On Line

A. Source Code

Tabel 1. Source Code `line.cpp` Modul 1

1	<code>#include "line.h"</code>
2	
3	<code>double gradient(const Line * l, const Point * p) {</code>
4	<code> return (l->a * p->x) + (l->b * p->y) + l->c;</code>
5	<code>}</code>
6	
7	<code>std::string CheckPointPosition(double gradient) {</code>
8	<code> if (gradient > EPSILON) {</code>
9	<code> return "Left";</code>
10	<code> } else if (gradient < -EPSILON) {</code>
11	<code> return "Right";</code>
12	<code> } else {</code>
13	<code> return "On Line";</code>
14	<code> }</code>
15	<code>}</code>

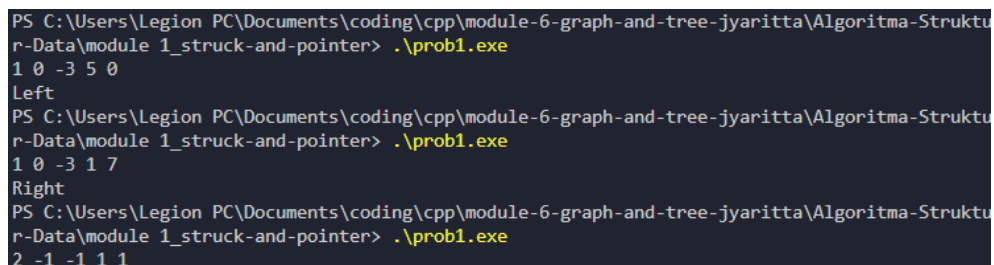
Tabel 2. Source Code prob1.cpp Modul 1

```

1 #include <iostream>
2 #include <string>
3 #include "line.h"
4 #include "point.h"
5
6 using namespace std;
7
8 int main() {
9     Line l1;
10    Point p1;
11
12    cin >> l1.a >> l1.b >> l1.c >> p1.x >> p1.y;
13
14    double hasil_evaluasi = gradient(&l1, &p1);
15    string status = CheckPointPosition(hasil_evaluasi);
16
17    cout << status << endl;
18
19    return 0;
20 }

```

B. Output Program



```

PS C:\Users\Legion PC\Documents\coding\cpp\module-6-graph-and-tree-jyaritta\Algoritma-Struktur-Data\module_1_struct-and-pointer> .\prob1.exe
1 0 -3 5 0
Left
PS C:\Users\Legion PC\Documents\coding\cpp\module-6-graph-and-tree-jyaritta\Algoritma-Struktur-Data\module_1_struct-and-pointer> .\prob1.exe
1 0 -3 1 7
Right
PS C:\Users\Legion PC\Documents\coding\cpp\module-6-graph-and-tree-jyaritta\Algoritma-Struktur-Data\module_1_struct-and-pointer> .\prob1.exe
2 -1 -1 1 1

```

Gambar 1. Screenshot Hasil Jawaban Soal 1 Modul 1

C. Pembahasan

Algoritma ini bekerja untuk menentukan posisi sebuah titik $P(x_p, y_p)$ terhadap suatu garis lurus dalam bentuk persamaan umum $ax + by + c = 0$.

Proses dimulai di dalam fungsi utama `main()` pada `prob1.cpp` untuk melakukan deklarasi dua buah objek struktur data, yaitu `l1` bertipe `Line` untuk koefisien garis dan `p1` bertipe `Point` untuk koordinat titik. Program menerima input dari pengguna melalui perintah `cin` secara berurutan untuk mengisi nilai variabel garis `l1.a`, `l1.b`, `l1.c`, dan koordinat titik `p1.x` dan `p1.y`.

Misalnya, input pertama ialah `1 0 -3 5 0` dimana variabel secara berurutan tersimpan ke `a`, `b`, `c`, `x` dan `y`. Jika kita mengerjakannya dengan rumus, maka:

$$\begin{aligned}
 ax + by + c &= 0 \\
 (1 * 5) + (0 * 0) - 3 &= 0 \\
 5 + 0 - 3 &= 0 \\
 2 &= 0
 \end{aligned}$$

Setelah data tersimpan, program memanggil fungsi `gradient` dari `line.cpp` untuk menghitung dengan mensubstitusikan koordinat titik ke persamaan garis dengan rumus matematika $(l \rightarrow a * p \rightarrow x) + (l \rightarrow b * p \rightarrow y) + l \rightarrow c$ dan hasil perhitungan ini disimpan di `hasil_evaluasi`.

Untuk menentukan status posisi spesifik dari titik tersebut program meneruskan variabel `hasil_evaluasi` ke fungsi `CheckPointPosition` pada `line.cpp` dimana fungsi ini melakukan pengecekan dengan struktur percabangan `if-else` bertingkat. Untuk mengantisipasi masalah presisi, pengecekan tidak menggunakan angka 0, tapi menggunakan `EPSILON` Epsilon sendiri adalah nilai positif sangat kecil yang digunakan sebagai batas toleransi untuk membandingkan dua bilangan *floating-point* agar menghindari kesalahan pembulatan.

Pengecekan pertama memeriksa apakah `gradient > EPSILON`. Jika terpenuhi, maka mengembalikan "Left", yang menandakan titik berada di sebelah kiri garis. Pengecekan Kedua, memeriksa apakah `gradient < -EPSILON`. Jika terpenuhi, mengembalikan string "Right" yang berarti titik berada di sebelah kanan garis. Kondisi Default dimana kedua kondisi di atas salah atau setara nol, antara `-EPSILON` dan `EPSILON`, maka program akan mengembalikan nilai "On Line", yang menyatakan bahwa titik terletak tepat pada garis.

SOAL 2

Diberikan sebuah lingkaran C dengan pusat (c_x, c_y) dan jari-jari $r > 0$, serta sebuah titik $P = (p_x, p_y)$. Tentukan posisi titik P terhadap lingkaran C dan mengimplementasikan fungsi `distance` dan `CheckPointInCircle` pada file header yang diberikan.

Input	Output
0 0 5 3 4	On Circle
0 0 5 1 1	Inside
0 0 5 4 4	Outside
-2 -2 4 2 -2	On Circle

A. Source Code

Tabel 3. Source Code `circle.cpp` Modul 1

```
1 #include "circle.h"
2 #include <cmath>
3
4 double distance(const Circle * c, const Point * p) {
5     return std::sqrt(std::pow(p->x - c->centre.x, 2) +
6     std::pow(p->y - c->centre.y, 2));
7 }
8
9 std::string CheckPointInCircle(double selisih) {
10     if (selisih < -EPSILON) {
11         return "Inside";
12     } else if (selisih > EPSILON) {
13         return "Outside";
14     } else {
15         return "On Circle";
16     }
17 }
```

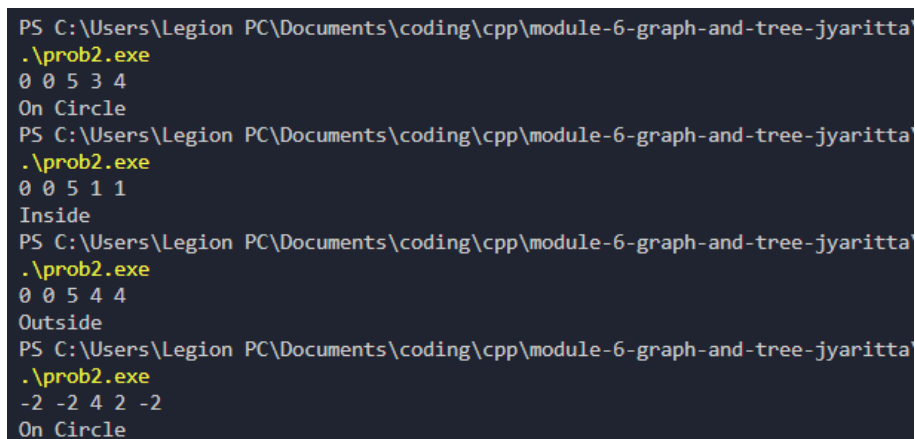
Tabel 4. Source Code prob2.cpp Modul 1

```

1  #include <iostream>
2  #include <string>
3  #include "circle.h"
4  #include "point.h"
5
6  using namespace std;
7
8  int main() {
9      Circle c;
10     Point p;
11
12     cin >> c.centre.x >> c.centre.y >> c.radius >> p.x >>
    p.y;
13
14     double d = distance(&c, &p);
15     double selisih = d - c.radius;
16     string status = CheckPointInCircle(selisih);
17     cout << status << endl;
18     return 0;
19 }

```

B. Output Program



```

PS C:\Users\Legion PC\Documents\coding\cpp\module-6-graph-and-tree-jyaritta>
.\prob2.exe
0 0 5 3 4
On Circle
PS C:\Users\Legion PC\Documents\coding\cpp\module-6-graph-and-tree-jyaritta>
.\prob2.exe
0 0 5 1 1
Inside
PS C:\Users\Legion PC\Documents\coding\cpp\module-6-graph-and-tree-jyaritta>
.\prob2.exe
0 0 5 4 4
Outside
PS C:\Users\Legion PC\Documents\coding\cpp\module-6-graph-and-tree-jyaritta>
.\prob2.exe
-2 -2 4 2 -2
On Circle

```

Gambar 2. Screenshot Hasil Jawaban Soal 2 Modul 1

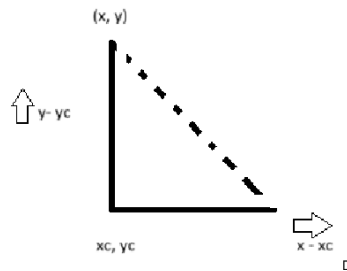
C. Pembahasan

Algoritma pada program ini bertujuan untuk menentukan posisi titik $P(x_p, y_p)$ terhadap sebuah lingkaran C yang memiliki titik pusat di (c_x, c_y) dan jari-jari r . Penggunaan library `<cmath>` untuk fungsi matematika bawaan C++, seperti `sqrt` atau akar kuadrat dan `pow` atau pangkat.

Fungsi `distance` pada `circle.cpp` menghitung jarak linier antara titik pusat lingkaran dengan titik P menggunakan prinsip Teorema Pythagoras. Prinsip tersebut diimplementasikan dengan `sqrt(std::pow(p->x - c->centre.x, 2) + std::pow(p->y - c->centre.y, 2))` dari $Jarak = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$. Nilai $(x_p - c_x)$ dan $(y_p - c_y)$

merepresentasikan selisih koordinat antara titik yang diuji dengan pusat lingkaran pada sumbu X dan Y.

Fungsi `CheckPointInCircle` mengevaluasi posisi titik berdasarkan nilai selisih menggunakan percabangan if-else dengan `EPSILON`, fungsi ini akan mengembalikan teks "Inside" jika selisih bernilai negatif lebih kecil dari `-EPSILON`, "Outside" jika selisih bernilai positif lebih besar dari `EPSILON`, dan "On Circle" jika nilainya netral atau tepat berada di garis lingkaran. Secara menyeluruh Nilai $(x-x_c)$ dan $(y-y_c)$ merepresentasikan selisih koordinat antara titik yang diuji dan pusat lingkaran pada sumbu X dan Y. Lingkaran sendiri ialah kumpulan titik dengan jarak sama ke pusat. Secara visual rumus jarak tadi berupa:



Sebagai contoh, input pertama ialah 0 0 5 3 4 dimana variabel secara berurutan tersimpan ke c. Jika kita mengerjakannya dengan rumus, maka:

$$\begin{aligned} d &= \sqrt{(x - x_1)^2 + (y - y_1)^2} \\ d &= \sqrt{(3 - 0)^2 + (4 - 0)^2} \\ d &= \sqrt{9 + 16} \\ d &= \sqrt{25} \\ d &= 5 \end{aligned}$$

Karena jarak titik P ke pusat adalah 5 begitu pula dengan radius, jika jaraknya sama persis dengan panjang jari-jarim, maka titik tersebut tidak berada di dalam dan tidak juga di luar, melainkan tepat menginjak garis lingkarannya.

MODUL 2: STACK AND QUEUE

SOAL 1: Implementasi Stack

Soal 1: Implementasi Stack Pada file `stack.h` yang diberikan, implementasikan seluruh fungsi yang dideklarasikan ke dalam file `stack.cpp`. Untuk menangani kondisi tidak valid (invalid case), gunakan mekanisme exception handling dengan `throw`.

Jelaskan alur algoritma dari setiap fungsi yang diimplementasikan, serta berikan justifikasi atau pembuktian singkat mengenai kebenaran algoritma tersebut. Selain itu, lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi yang ada. (Gunakan Big-O Notation).

A. Source Code

Tabel 5. Source code *stack.cpp* Modul 2

1	#include "stack.h"
2	#include <stdexcept>
3	
4	void init(Stack* s){
5	s->top = nullptr;
6	}
7	
8	bool isEmpty(const Stack* s){
9	return s->top == nullptr;
10	}
11	
12	bool isFull(const Stack* s){
13	if (isEmpty(s)) {
14	return false;
15	}
16	return s->top == &(s->data[MAX - 1]);
17	}
18	
19	void push(Stack* s, int value) {
20	if (isFull(s)) {
21	throw std::overflow_error("Penuh");
22	}
23	
24	if (isEmpty(s)){
25	s->top = &(s->data[0]);
26	} else {
27	s->top++;
28	}
29	*(s->top) = value;
30	}
31	
32	void pop(Stack* s){
33	if (isEmpty(s)) {
34	throw std::underflow_error("Kosong");
35	}
36	if (s->top == &(s->data[0])) {
37	s->top = nullptr;
38	} else {
39	s->top--;
40	}
41	}
42	
43	int peek(const Stack* s){
44	if (isEmpty(s)) {
45	throw std::underflow_error("Kosong");
46	}
47	return *(s->top);
48	}

B. Pembahasan

Pada baris [1 sampai 2] `stack.cpp`, mengimpor file header "`stack.h`" untuk mengenali `struct Stack` dan mengimpor library `<stdexcept>` untuk memunculkan pesan error dengan `throw`.

Pada baris [4 sampai 6] `stack.cpp`, fungsi `init(Stack* s)` menginisialisasi tumpukan kosong dengan mengatur pointer `top` menjadi `nullptr`.

Pada baris [8 sampai 10] `stack.cpp`, terdapat fungsi `isEmpty(const Stack* s)` bertipe boolean dimana akan mengembalikan nilai `true` jika pointer `top` menunjuk ke `nullptr` atau pointer kosong yang tidak menunjuk ke alamat memori manapun.

Pada baris [12 sampai 17] `stack.cpp`, fungsi `isFull(const Stack* s)` mengecek apakah kapasitas array sudah penuh dengan membandingkan alamat pointer `top` terhadap alamat elemen terakhir array `&(s->data[MAX - 1])`. Mengapa harus `[MAX - 1]` dan tidak langsung `MAX` saja? Ini karena komputer mulai menghitung dari angka 0, bukan angka 1 atau konsep ini disebut Zero-Based Indexing.

Pada baris [19 sampai 31] `stack.cpp`, fungsi `push(Stack* s, int value)` berfungsi untuk memasukkan data baru. Dimana pertama-tama mengecek apakah batas memori masih cukup. Jika iya, pointer `top` akan digeser naik dan nilai baru dimasukkan.

Pada baris [33 sampai 44] `stack.cpp`, fungsi `pop(Stack* s)` berfungsi untuk menghapus data paling atas. Pertama-tama fungsi ini akan melakukan `throw std::underflow_error("Kosong")` jika tumpukan sudah kosong. Jika ada isinya, fungsi ini akan menghapus data dengan menggeser pointer `top` turun satu Tingkat `s->top--`.

Pada baris [46 sampai 52] `stack.cpp`, terdapat fungsi `peek(const Stack* s)` yang berfungsi untuk mengembalikan nilai data di posisi paling atas tumpukan tanpa mengubah posisi pointer dan menghapusnya dengan mengakses nilai yang ditunjuk oleh operator pointer `*(s->top)`, angka di atas tumpukan tetap ada dan tidak terhapus.

SOAL 2: Si Palui dan Mesin Hitung Aneh

Time Limit: 1.0 s
Memory Limit: 64 MB

Si Palui menemukan sebuah mesin hitung tua di gudang. Mesin tersebut memiliki cara kerja yang tidak biasa. Alih-alih menggunakan format perhitungan seperti yang biasa kita kenal, mesin ini membaca input berupa deretan simbol yang dipisahkan oleh spasi.

Setiap simbol bisa berupa:

- sebuah bilangan bulat, atau
- sebuah operator (+, -, *, /)
- Cara kerja mesin adalah sebagai berikut:
- Mesin membaca simbol dari kiri ke kanan.
- Jika simbol adalah bilangan, maka bilangan tersebut disimpan ke dalam memori mesin.
- Jika simbol adalah operator, maka mesin akan mengambil dua nilai yang paling baru dimasukkan ke dalam memori yang disimpan, melakukan operasi sesuai operator tersebut, lalu menyimpan kembali hasilnya ke dalam memori.

Setelah seluruh simbol diproses, akan tersisa satu nilai di dalam memori. Nilai inilah yang ditampilkan sebagai hasil akhir. Sebagai asisten Si Palui, tugas Anda adalah membantu menghitung hasil dari mesin tersebut.

Batasan

- $1 \leq n \leq 100$
- $-1000 \leq A_i \leq 1000$

Keterangan

- n adalah jumlah simbol dalam ekspresi
- A_i adalah nilai bilangan pada input
- Simbol yang valid hanya:
- bilangan bulat
- operator: +, -, *, /

Format Input

n

A1 A2 ... An

Keterangan:

- Baris pertama n menyatakan jumlah atau panjang simbol
- Baris kedua berisi simbol A sebanyak n buah

Contoh Kasus

Input	Output
3 2 3 +	5
9 5 1 2 + 4 * + 3 -	14
5 10 2 / 3 *	15

A. Source Code

Tabel 6. Source code prob1.cpp Modul 2

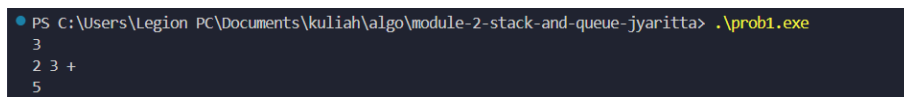
1	#include <iostream>
2	#include <string>
3	#include "stack.h"
4	
5	using namespace std;
6	
7	int main() {
8	int n;
9	if (!(cin >> n)) return 0;
10	
11	Stack memory;
12	init(&memory);
13	
14	for (int i = 0; i < n; i++) {
15	string token;
16	cin >> token;
17	
18	if (token == "+" token == "-" token == "*" token == "/") {
19	int A2 = peek(&memory);
20	pop(&memory);
21	
22	int A1 = peek(&memory);
23	pop(&memory);
24	
25	int hasil = 0;
26	
27	if (token == "+") {
28	hasil = A1 + A2;
29	} else if (token == "-") {
30	hasil = A1 - A2;

```

31         } else if (token == "*") {
32             hasil = A1 * A2;
33         } else if (token == "/") {
34             hasil = A1 / A2;
35         }
36
37         push(&memory, hasil);
38
39     } else {
40         push(&memory, stoi(token));
41     }
42 }
43
44 cout << peek(&memory) << endl;
45
46 return 0;
47 }

```

B. Output Program

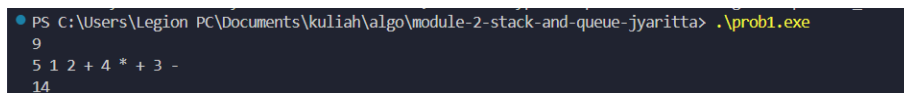


```

PS C:\Users\Legion PC\Documents\kuliah\algo\module-2-stack-and-queue-jyaritta> .\prob1.exe
3
2 3 +
5

```

Gambar 3. Screenshot 1 Hasil Jawaban Soal 1 Modul 2

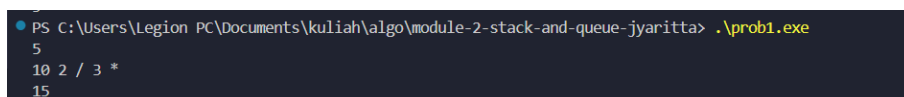


```

PS C:\Users\Legion PC\Documents\kuliah\algo\module-2-stack-and-queue-jyaritta> .\prob1.exe
9
5 1 2 + 4 * + 3 -
14

```

Gambar 4. Screenshot 2 Hasil Jawaban Soal 1 Modul 2



```

PS C:\Users\Legion PC\Documents\kuliah\algo\module-2-stack-and-queue-jyaritta> .\prob1.exe
5
10 2 / 3 *
15

```

Gambar 5. Screenshot 3 Hasil Jawaban Soal 1 Modul 2

C. Pembahasan

Pada baris [1 sampai 3] `prob1.cpp`, syntax `#include <iostream>` untuk mengimpor input/output, `<string>` untuk bisa manipulasi string/teks, dan `"stack.h"` agar program utama mengenali fungsi yang telah dibuat.

Pada baris [5] `prob1.cpp`, syntax `using namespace std;` digunakan agar tidak menuliskan ulang `std::` lagi pada perintah C++.

Pada baris [7 sampai 54] `prob1.cpp`, terdapat fungsi utama `int main()` yang merupakan titik awal program akan pertama kali dieksekusi oleh komputer.

Pada baris [8 sampai 10] `prob1.cpp`, program mendeklarasikan variabel `n` bertipe `integer` untuk membaca jumlah ukuran yang diinputkan user pada baris pertama.

Pada baris [12 sampai 13] `prob1.cpp`, program membuat variabel memory bertipe `Stack` dan langsung memasukkan alamat memorinya ke dalam fungsi `init(&memory)` agar tumpukannya disiapkan dalam keadaan kosong.

Pada baris [16 sampai 48] `prob1.cpp`, terdapat perulangan `for` yang berjalan sebanyak `n` kali untuk membaca input berupa `string token`. Jika `token` berupa operator `(+, -, *, /)`, program akan memanggil fungsi `pop` dua kali untuk mengambil angka `A1` dan `A2` dari tumpukan memori.

Pada baris [33 sampai 43] `prob1.cpp`, terdapat implementasi logika percabangan aritmatika dimana program mengecek isi variabel `token`. Jika `token` berisi simbol penjumlahan, pengurangan, perkalian, atau pembagian, maka program akan memproses variabel `A1` dan `A2` sesuai operasi aritmatika. Khusus untuk pembagian, hasil akan dibulatkan sesuai sifat integer pada C++. Hasil operasi tadi didorong kembali ke dalam stack memori dan memastikan nilai tersebut siap dieksekusi jika terdapat operator lain di belakangnya.

Pada baris [45 sampai 48] `prob1.cpp`, pada `syntax else`, jika simbol `token` yang dibaca adalah angka maka program akan mengubah teks tersebut menjadi angka `integer` menggunakan fungsi `stoi()` dan memasukkannya ke tumpukan.

Pada baris [51] `prob1.cpp`, `syntax cout` berguna untuk mencetak satu angka terakhir yang tersisa di dalam memori dengan fungsi `peek(&memory)`.

Pada baris [53] `prob1.cpp`, terdapat `syntax return 0;` untuk mengakhiri fungsi `main` bahwa program telah berjalan dan selesai tanpa adanya error.

SOAL 3: Implementasi Queue

Pada file queue.h yang diberikan, implementasikan seluruh fungsi yang dideklarasikan ke dalam file queue.cpp. Untuk menangani kondisi tidak valid (invalid case), gunakan mekanisme exception handling dengan throw.

Jelaskan alur algoritma dari setiap fungsi yang diimplementasikan, serta berikan justifikasi atau pembuktian singkat mengenai kebenaran algoritma tersebut. Selain itu, lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi yang ada. (Gunakan Big-O Notation).

A. Source Code

Tabel 7. Source code queue.cpp Modul 2

1	#include "queue.h"
2	#include <stdexcept>
3	
4	void init(Queue* q) {
5	q->front = nullptr;
6	q->rear = nullptr;
7	}
8	
9	bool isEmpty(const Queue* q) {
10	return q->front == nullptr;
11	}
12	
13	bool isFull(const Queue* q) {
14	if (isEmpty(q)) return false;
15	return q->rear == &(q->data[MAX - 1]);
16	}
17	
18	void enqueue(Queue* q, int value) {
19	if (isFull(q)) {
20	throw std::overflow_error("Penuh");
21	}
22	
23	if (isEmpty(q)) {
24	q->front = &(q->data[0]);
25	q->rear = &(q->data[0]);
26	} else {
27	q->rear++;
28	}
29	*(q->rear) = value;
30	}
31	void dequeue(Queue* q) {
32	if (isEmpty(q)) {
33	throw std::underflow_error("Kosong");
34	}
35	
36	if (q->front == q->rear) {
37	q->front = nullptr;
38	q->rear = nullptr;

```

39     } else {
40         q->front++;
41     }
42 }
43
44 int front(const Queue* q) {
45     if (isEmpty(q)) {
46         throw std::underflow_error("Kosong");
47     }
48     return *(q->front);
49 }
50
51 int back(const Queue* q) {
52     if (isEmpty(q)) {
53         throw std::underflow_error("Kosong");
54     }
55     return *(q->rear);
56 }

```

B. Pembahasan

Pada baris [1 sampai 2] `queue.cpp`, mengimpor file header `"queue.h"` untuk mengenali struct `queue` dan mengimpor library `<stdexcept>` untuk memunculkan pesan error dengan `throw`.

Pada baris [4 sampai 7] `queue.cpp`, fungsi `init(Queue* q)` berfungsi untuk menginisialisasi antrean menjadi kosong dengan cara membuat pointer `front` (depan) dan `rear` (belakang) menunjuk `nullptr`.

Pada baris [9 sampai 11] `queue.cpp`, fungsi `isEmpty(const Queue* q)` berfungsi untuk memvalidasi apakah antrean kosong dengan cara melihat pointer `front` apakah itu bernilai `nullptr`. Jika iya, fungsi mengembalikan nilai `true`.

Pada baris [13 sampai 16] `queue.cpp`, fungsi `isFull(const Queue* q)` digunakan untuk memvalidasi batas memori array. Antrean akan dinyatakan penuh atau `true` jika pointer `rear` telah berada pada bagian paling akhir array `&(q->data[MAX - 1])`.

Pada baris [18 sampai 30] `queue.cpp`, fungsi `enqueue(Queue* q, int value)` berfungsi untuk memasukkan elemen baru dari bagian belakang antrean. Fungsi diawali dengan memanggil `isFull`, jika batas tercapai, program akan memicu `throw std::overflow_error()`. Jika masih ada memori array, pointer `front` mengarah ke indeks 0 atau deretan paling pertama dan pointer `rear` akan mengarah ke indeks 0 atau deretan paling akhir. Jika ada data masuk, pointer `rear` akan bergeser naik (`rear++`).

Pada baris [31 sampai 41] `queue.cpp`, `dequeue(Queue* q)` digunakan untuk mengeluarkan elemen paling depan. Di dalam fungsi ini terdapat `isEmpty` dimana akan melakukan `throw std::underflow_error()` jika antrean kosong. Jika elemen sudah tersisa satu `front == rear`, pointer dikembalikan ke `nullptr`. Jika elemen lebih dari satu, pointer `front` digeser naik untuk mengabaikan data terdepan yang lama.

Pada baris [43 sampai 55] `queue.cpp`, terdapat fungsi `front()` dan `back()` yang masing-masing berfungsi untuk melihat nilai dengan menggunakan operator dereference atau tanda `*` dari elemen paling depan dan paling belakang pada antrean tanpa mengubah struktur pointer, dilengkapi dengan penanganan error jika antrean kosong.

SOAL 4: Si Palui dan Deretan Angka

Si Palui memiliki sebuah deretan angka yang tersusun rapi. Ia tertarik untuk mengamati sebagian angka secara berurutan dengan jumlah tertentu.

Ia memilih sejumlah k angka yang bersebelahan, lalu menghitung jumlahnya. Setelah itu, ia menggeser pilihannya satu langkah ke kanan dan kembali menghitung jumlah dari angka-angka yang terpilih.

Proses ini dilakukan terus hingga tidak memungkinkan lagi untuk memilih k angka berurutan. Sebagai asistennya, bantulah Si Palui untuk menentukan hasil perhitungan tersebut.

Batasan

- $1 \leq n \leq 100$
- $-1 \leq k \leq n$
- $-1000 \leq A_i \leq 1000$

Keterangan:

- n : jumlah elemen
- k : ukuran jendela
- A_i : elemen array

Format Input

n k

A_1 A_2 ... A_n

Keterangan:

- Baris pertama n menyatakan jumlah atau panjang array. Lalu k besar atau jumlah elemen dalam satu kelompok yang dibentuk
- Baris kedua berisi simbol A sebanyak n buah

Format Output

Cetak hasil tiap masing-masing kelompok dalam satu baris dipisahkan dengan spasi

Contoh Kasus

Input	Output
5 3 1 2 3 4 5	6 9 12
5 5 1 2 3 4	15
6 2 4 -1 2 -3 5 1	3 1 -1 2 6

A. Source Code

Tabel 8. Source code prob2.cpp Modul 2

```
1 #include <iostream>
2 #include "queue.h"
3
4 using namespace std;
5
6 int main() {
7     int n, k;
8     if (!(cin >> n >> k)) return 0;
9     Queue window;
10    init(&window);
11    long long current_sum = 0;
12    int temp;
13
14    for (int i = 0; i < k; i++) {
15        cin >> temp;
16        enqueue(&window, temp);
17        current_sum += temp;
18    }
19    cout << current_sum;
20
21    for (int i = k; i < n; i++) {
22        cin >> temp;
23        current_sum -= front(&window);
24        ~ dequeue(&window);
25        enqueue(&window, temp);
26        current_sum += temp;
27        cout << " " << current_sum;
28    }
29    cout << endl;
30    return 0;
31 }
```

B. Output Program

```
PS C:\Users\Legion PC\Documents\kuliah\algo\module-2-stack-and-queue-jyaritta> .\prob2.exe
5 3
1 2 3 4 5
6 9 12
```

Gambar 6. Screenshot 1 Hasil Jawaban Soal 2 Modul 2

```
PS C:\Users\Legion PC\Documents\kuliah\algo\module-2-stack-and-queue-jyaritta> .\prob2.exe
5 5
1 2 3 4 5
15
```

Gambar 7. Screenshot 2 Hasil Jawaban Soal 2 Modul 2

```
PS C:\Users\Legion PC\Documents\kuliah\algo\module-2-stack-and-queue-jyaritta> .\prob2.exe
6 2
4 -1 2 -3 5 1
3 1 -1 2 6
```

Gambar 8. Screenshot 3 Hasil Jawaban Soal 2 Modul 2

C. Pembahasan

Pada baris [1 sampai 4] `prob2.cpp`, syntax `#include <iostream>` untuk mengimpor input/output dan memanggil header `"queue.h"` agar program utama mengenali fungsi yang telah dibuat.

Pada baris [6 sampai 8] `prob2.cpp`, terdapat deklarasi fungsi utama `int main()` serta inisialisasi variabel `n` untuk jumlah total elemen array dan `k` untuk ukuran jendela geser, dan pembacaan input dari pengguna untuk elemen `n` dan `k`.

Pada baris [9 sampai 12] `prob2.cpp`, program mendeklarasikan variabel `window` dari tipe data `Queue` dan melakukan inisialisasi menggunakan `init(&window)`. mendeklarasikan pula variabel `current_sum` bertipe `long long` untuk kapasitas perhitungan jika nantinya ada akumulasi angka bernilai besar.

Pada baris [14 sampai 18] `prob2.cpp`, program mengeksekusi langkah pertama dari algoritma menggunakan perulangan `for`. Program memakai `enqueue` sebanyak `k` angka pertama ke dalam antrian, lalu langsung menjumlahkannya ke dalam variabel `current_sum`.

Pada baris [19] `prob2.cpp`, program mencetak `current_sum` yang merupakan total dari jendela pertama ke layar terminal.

Pada baris [21 sampai 28] `prob2.cpp`, program masuk ke langkah selanjutnya yaitu setiap iterasi mulai dari indeks `ke-k` hingga `n`, program akan mengurangi nilai paling depan dari total saat ini atau `current_sum -= front(&window)`, lalu mengeluarkan elemen tersebut dari memori dengan `dequeue`.

Pada baris [25 sampai 27] `prob2.cpp`, secara bersamaan program membaca satu angka baru yang masuk ke dalam `window`, menyimpannya ke antrian bagian belakang atau `enqueue`, lalu menambahkannya ke perhitungan `current_sum`, kemudian langsung mencetak hasil perhitungan yang baru yang akan dipisah dengan spasi agar angka hasil perhitungan sejajar ke samping dalam satu baris.

Pada baris [29 sampai 31] `prob2.cpp`, program memberikan baris baru di akhir keluaran dengan `endl` sekaligus membersihkan buffer keluaran dan menutup fungsi utama dengan `return 0`.

Penggambaran langkah-langkah window pada percobaan soal 2A adalah sebagai berikut.

1	$1 + 2 + 3 = 6$
2	$6 - 1 + 4 = 9$
3	$9 - 2 + 5 = 12$

MODUL 3: LINKED LIST

SOAL 1: Implementasi Senarai Berantai Tunggal Melingkar

General Technical Requirements:
Standard: C++23
Memory Management: Manual (new / delete)
Constraints: Strictly NO STL (<vector>, <list>, dll), NO Global Arrays.

Pada file `single_linked_list.h` yang diberikan, implementasikan struktur `SingleLinkedList` beserta fungsi-fungsinya ke dalam file `single_linked_list.cpp`.

Jelaskan alur algoritma dari setiap metode yang diimplementasikan. Pastikan metode `clear()` berjalan dengan benar untuk menghindari memory leak (kebocoran memori). Lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi menggunakan Big-O Notation. Seluruh manajemen memori harus dialokasikan secara manual di dalam heap.

Jelaskan menggunakan alur bagaimana dan mengapa. Jelaskan kenapa algoritma yang kalian rancang itu valid dan yakin benar.

A. Source Code

Tabel 9. Source code `single_linked_list.cpp` Modul 3

1	<code>#include "single_linked_list.h"</code>
2	<code>#include <iostream></code>
3	<code>#include <stdexcept></code>
4	
5	<code>void SingleLinkedList::init() {</code>
6	<code> head = new Node;</code>
7	<code> head->data = 0;</code>
8	<code> head->next = head;</code>
9	<code> tail = head;</code>
10	<code> size = 0;</code>
11	<code>}</code>
12	
13	<code>bool SingleLinkedList::is_empty() {</code>
14	<code> return size == 0;</code>
15	<code>}</code>
16	
17	<code>void SingleLinkedList::add_front(int val) {</code>
18	<code> Node* newNode = new Node;</code>
19	<code> newNode->data = val;</code>
20	<code> newNode->next = head->next;</code>
21	<code> head->next = newNode;</code>
22	
23	<code> if(size == 0) {</code>

```

24         tail = newNode;
25     }
26     size++;
27 }
28
29 void SingleLinkedList::add_back(int val){
30     Node* newNode = new Node;
31     newNode->data = val;
32     newNode->next = head;
33
34     tail->next = newNode;
35     tail = newNode;
36     size++;
37 }
38
39 void SingleLinkedList::add_idx(int val, int idx){
40     if (idx < 0 || idx > size) throw
std::out_of_range("Index tidak valid");
41     if (idx == 0) {
42         add_front(val);
43     } else if (idx == size) {
44         add_back(val);
45     } else {
46         Node* prev = head;
47         for (int i = 0; i < idx; i++) {
48             prev = prev->next;
49         }
50         Node* newNode = new Node;
51         newNode->data = val;
52         newNode->next = prev->next;
53         prev->next = newNode;
54         size++;
55     }
56 }
57
58 void SingleLinkedList::delete_front(){
59     if (is_empty()) return;
60
61     Node* temp = head->next;
62     head->next = temp->next;
63
64     if (size == 1) {
65         tail = head;
66     }
67     tail->next = head;
68     delete temp;
69     size--;
70 }
71
72 void SingleLinkedList::delete_back(){
73     if (is_empty()) return;
74

```

```

75     Node* prev = head;
76     while (prev->next != tail) {
77         prev = prev->next;
78     }
79     Node* temp = tail;
80     prev->next = head;
81     tail = prev;
82
83     delete temp;
84     size--;
85 }
86
87
88 void SingleLinkedList::delete_idx(int idx){
89     if (idx < 0 || idx >= size) throw
std::out_of_range("Index tidak valid");
90     if (idx == 0) {
91         delete_front();
92     } else if (idx == size - 1) {
93         delete_back();
94     }
95     else {
96         Node* prev = head;
97         for (int i = 0; i < idx; i++) {
98             prev = prev->next;
99         }
100
101         Node* temp = prev->next;
102         prev->next = temp->next;
103
104         delete temp;
105         size--;
106     }
107 }
108
109 void SingleLinkedList::display(){
110     if (is_empty()) {
111         std::cout << "List kosong\n";
112         return;
113     }
114     Node* curr = head->next;
115     while (curr != head) {
116         std::cout << curr->data << " ";
117         curr = curr->next;
118     }
119     std::cout << "\n";
120 }
121
122 int SingleLinkedList::get(int idx){
123     if (idx < 0 || idx >= size) throw
std::out_of_range("Index tidak valid");
124     Node* curr = head->next;

```

```

125     for (int i = 0; i < idx; i++) {
126         curr = curr->next;
127     }
128     return curr->data;
129 }
130
131
132 void SingleLinkedList::set(int val, int idx){
133     if (idx < 0 || idx >= size) throw
std::out_of_range("Index tidak valid");
134     Node* curr = head->next;
135     for (int i = 0; i < idx; i++) {
136         curr = curr->next;
137     }
138     curr->data = val;
139 }
140
141 void SingleLinkedList::clear(){
142     while (!is_empty()) {
143         delete_front();
144     }
145 }

```

B. Pembahasan

Inisialisasi fungsi:

1. `Init()`

berfungsi untuk menetapkan nilai awal pada variable agar tidak bernilai kosong, atau node pertama, elemen dasar yang menyimpan data dan pointer ke node lainnya. kompleksitas Waktu $O(1)$ karena hanya mengatur nilai beberapa variabel awal.

2. `is_empty()`

berfungsi untuk mengecek apakah list tersebut kosong. Kompleksitas Waktu $O(1)$ karena hanya mengecek satu variable.

3. `add_front()`

berfungsi untuk menambahkan data di posisi paling depan setelah Dummy Head atau simpul tambahan yang ditempatkan di bagian paling awal sebuah struktur data agar mudah dimanipulasi. Kalau list sebelumnya kosong, node baru ini akan menjadi tail. Kompleksitas Waktu $O(1)$ karena hanya memasukkan node baru. Kompleksitas Waktu $O(1)$ karena hanya mengecek satu variable.

4. `add_back()`

berfungsi untuk menambahkan data di posisi paling belakang dengan cara membuat node baru dan mengisi nilainya lalu memperbarui variabel tail ke node paling belakang yang baru. Kompleksitas Waktu tetap $O(1)$ karena satu kondisi if tidak membatalkan status kompleksitas konstan $O(1)$ dimana Status $O(1)$ baru akan gugur kalau ada perulangan.

5. `add_idx()`

berfungsi untuk menyisipkan data baru pada index dengan validasi if ($idx < 0 \parallel idx > size$) untuk mengecek terlebih dahulu apakah index yang diminta sesuai dengan aturan. Jika tidak, program akan melempar pesan error(`throw std::out_of_range` dan mencegah program melanjutkan proses agar tidak terjadi crash Kondisi Awal if ($idx == 0$)): Jika targetnya adalah index atau paling depan, fungsi takan mendelegasikan tugas dengan memanggil fungsi `add_front(val)`.

6. `delete_front()`

berfungsi untuk menghapus node di index spesifik. Target yang sudah dihapus ini kemudian dihapus dari memori komputer atau delete tem).

7. `delete_front()`

berfungsi untuk menghapus data paling depan setelah head guna menghindari memory leaks. Dengan command `head->next = temp->next;`, panah dari head langsung berpindah ke node setelah temp. Fungsi juga memastikan jika node yang dihapus itu adalah satu-satunya data. Karena temp sudah tidak masuk dalam rangkaian list, node ini segera dihancurkan dari Heap agar tidak terjadi memory leaks.

8. `delete_back()`

berfungsi untuk menghapus node di posisi paling ekor atau belakang guna menghindari memory leaks. Pertama fungsi juga akan mengecek apakah list kosong dengan `if is_empty()` lalu melakukan return agar tidak terjadi error. Jika syarat diluar daripada itu, karena ini adalah Single Linked List fungsi harus melakukan perulangan `while` untuk mencari node yang letaknya tepat sebelum ekor, tetapi aku lupa dengan command `prev->next = head;`, panah dari node `prev` langsung berpindah melingkar kembali ke `head` untuk memutus `temp`, dan `prev` diresmikan menjadi `tail` yang baru. Setelah itu, `delete temp;`. Karena `temp` sudah tidak masuk dalam rangkaian list, node ini segera dihancurkan dari Heap agar tidak terjadi memory leaks.

9. `delete_idx()`

berfungsi untuk menghapus node guna menghindari memory leaks. Pertama fungsi akan mengecek apakah indeks yang akan dihapus logis dengan `if (idx < 0 || idx >= size)` lalu melakukan throw karena indeks mulai dari 0 dan data yang diberikan ada 5 maka indeks maksimalnya ialah 4, jika melakukan `delete idk ke-5` maka akan terjadi error. Jika indeks berada di urutan paling akhir maka fungsi akan memanggil `delete_back()` untuk menghapus ekor. contoh kasus, Jika syarat diluar daripada itu maka akan melakukan `else` dimana akan indeks yang akan dihapus akan ditaruh ke variable sementara atau `temp`. lalu dengan command `prev->next = temp->next;` langsung berpindah ke node setelah `temp`. Setelah itu, `delete temp;` Karena `temp` sudah tidak masuk dalam rangkaian list, node ini segera dihancurkan dari Heap agar tidak terjadi memory leaks dan terakhir, ukuran `size` dikurangi.

10. `display()`

berfungsi untuk menampilkan seluruh isi data di dalam list ke terminal. Pertama fungsi akan mengecek apakah list tersebut sedang kosong dengan `if (is_empty())` dan mencetak teks "List kosong" lalu melakukan return agar program berhenti mengeksekusi sisa baris di bawahnya. Jika list ada isinya, fungsi akan membuat variabel penunjuk bernama `curr` atau `current` yang mulai berdiri di node pertama asli `head->next`. Karena list ini bersifat melingkar maka perulangan `while curr != head` akan terus berjalan mencetak nilai `curr->data`, sampai `curr` melingkar penuh dan kembali menyentuh `head`.

11. `get()`

berfungsi untuk mengambil atau membaca nilai pada posisi indeks tertentu tanpa menghapusnya dari list. Program menggunakan perulangan `for` untuk menyusuri list langkah demi langkah. Setelah berjalan sebanyak `idx` dan `curr` berhenti tepat di node yang diminta, fungsi langsung mengembalikan nilai dari node tersebut ke program utama dengan command `return curr->data;`

12. `set()`

berfungsi untuk mengubah atau menimpa nilai data pada posisi (indeks) tertentu dengan nilai (`val`) yang baru. Jika indeksnya benar, penunjuk `curr` akan mulai dari `head->next` lalu menyusuri list menggunakan perulangan `for` sampai menemukan node target. Fungsi tidak mengembalikan nilai, melainkan langsung menimpa data lama di dalam node tersebut dengan data yang baru melalui command `curr->data = val;`

13. `clear()`

berfungsi untuk membersihkan semua node yang ada di dalam list guna menghindari memory leaks. fungsi ini akan terus-menerus memanggil fungsi `delete_front()`; sampai list tersebut benar-benar kosong.

SOAL 2: Si Palui dan Mesin Hitung Aneh

Time Limit: 1.0 s
Memory Limit: 64 MB

Si Palui terjebak di dalam sebuah kuil kuno yang dikelilingi oleh N buah batu relikui yang membentuk formasi lingkaran sempurna. Untuk membuka pintu keluar, ia harus menghancurkan batu-batu tersebut dengan urutan tertentu menggunakan pancaran energi beruntun.

Pancaran energi bermula dari batu pertama, lalu melompat sejauh K langkah ke arah depan, lalu menghancurkan batu target tersebut. Namun, kuil ini memiliki anomali energi dinamis:

- Jika nilai ukiran pada batu yang baru saja dihancurkan adalah bilangan genap, maka lompatan berikutnya akan bertambah jauh ($K=K+1$).
- Jika nilai ukiran pada batu yang dihancurkan adalah bilangan ganjil, maka lompatan berikutnya akan melemah ($K=K-1$).
- Jika pada suatu titik energi lompatan habis atau negatif ($K \leq 0$), sistem akan mengalami reset dan mengembalikan nilai lompatan ke nilai K awal (saat pancaran energi pertama kali diinisialisasi).

Setelah seluruh batu hancur kecuali satu, batu terakhir itulah yang akan menjadi kunci pintu keluar.

Batasan

- $1 \leq N \leq 104$
- $1 \leq K \leq 109$
- $1 \leq A_i \leq 106$ (Nilai ukiran pada batu)
- Program wajib menggunakan implementasi struktur data yang dibuat dari soal 1.

Keterangan

Format Input

N K

A_1 A_2 ... A_N

Format Output

Sebuah bilangan bulat tunggal yang merupakan nilai ukiran pada batu terakhir yang tersisa

Contoh Kasus

Input	Output
5 2 1 4 3 6 2	6
4 5 10 11 12 13	12

A. Source Code

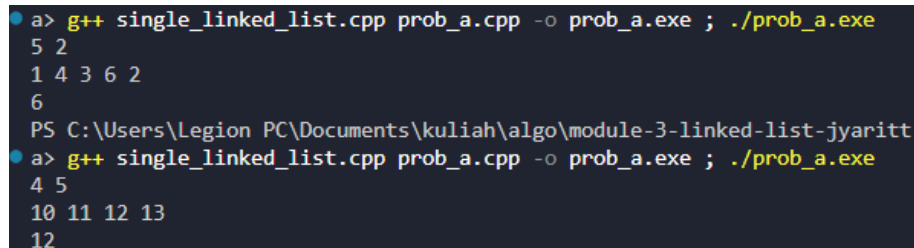
Tabel 10. Source code prob_a.cpp Modul 3

```

1  #include "single_linked_list.h"
2  #include <iostream>
3  using namespace std;
4
5  int main() {
6      int n, k;
7      if (!(cin >> n >> k)) return 0;
8
9      SingleLinkedList sll;
10     sll.init();
11
12     for (int i = 0; i < n; i++) {
13         int val;
14         cin >> val;
15         sll.add_back(val);
16     }
17
18     int k_awal = k;
19     int current_idx = 0;
20
21     while (sll.size > 1) {
22         current_idx = (current_idx + k - 1) % sll.size;
23
24         int batu_hancur = sll.get(current_idx);
25         sll.delete_idx(current_idx);
26
27         if (batu_hancur % 2 == 0) {
28             k++;
29         } else {
30             k--;
31         }
32
33         if (k <= 0) {
34             k = k_awal;
35         }
36     }
37
38     cout << sll.get(0) << "\n";
39     sll.clear();
40     return 0;
41 }

```

B. Output Program



```
a> g++ single_linked_list.cpp prob_a.cpp -o prob_a.exe ; ./prob_a.exe
5 2
1 4 3 6 2
6
PS C:\Users\Legion PC\Documents\kuliah\algo\module-3-linked-list-jyaritt
a> g++ single_linked_list.cpp prob_a.cpp -o prob_a.exe ; ./prob_a.exe
4 5
10 11 12 13
12
```

Gambar 9. Screenshot Hasil Jawaban Soal 1 Modul 3

C. Pembahasan

Pada awal program, `cin >> n >> k` digunakan untuk membaca dua input dari pengguna, yaitu `n` sebagai jumlah total batu dan `k` sebagai jumlah langkah awal. Pengecekan `if (!...)` berfungsi untuk menghentikan program apabila input yang dimasukkan tidak valid atau kosong. Setelah itu, `sll.init()`; dipanggil untuk menyiapkan struktur Single Linked List sebagai tempat data batu

Selanjutnya, program meminta input data batu melalui perulangan `for (int i = 0; i < n; i++)`. Setiap data batu yang dibaca dari input ditambahkan ke bagian belakang list menggunakan fungsi `sll.add_back(val)`. Selanjutnya ada `k_awal = k`; untuk menyimpan cadangan nilai langkah awal dan `current_idx = 0`; untuk menentukan titik awal perhitungan pada indeks pertama sebelum permainan dimulai.

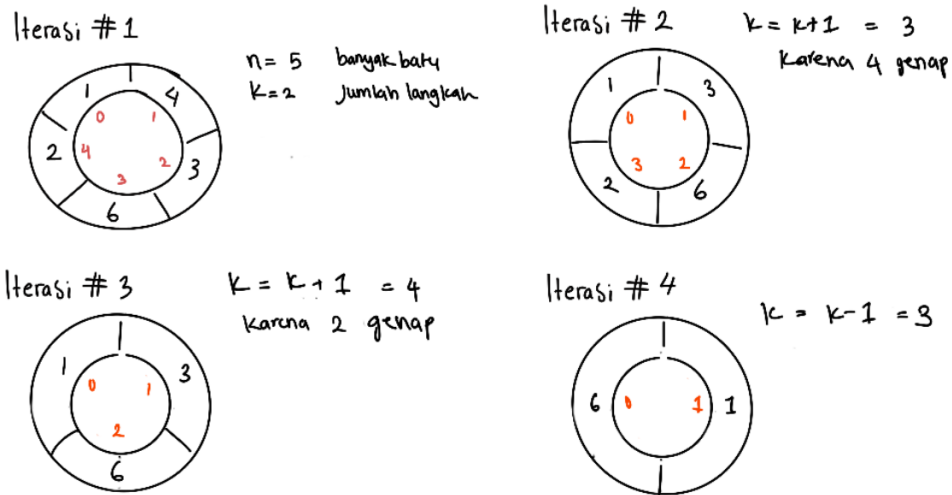
Selama jumlah batu di dalam list masih lebih dari satu, seluruh logika permainan akan terus dieksekusi berulang kali. Program akan menentukan target batu yang akan dihancurkan menggunakan rumus `current_idx = (current_idx + k - 1) % sll.size`;

Setelah indeks ditemukan, `sll.get(current_idx)`; mengambil dan menyimpan nilai sementara batu yang akan dihancurkan ke variabel `batu_hancur`, kemudian batu tersebut dihapus dari list menggunakan `sll.delete_idx(current_idx)`;

Berikutnya, program memperbarui aturan langkah dengan kondisi ganjil dan genap menggunakan modulo dan kondisi `if (batu_hancur % 2 == 0)`. Jika genap, jumlah langkah untuk putaran berikutnya akan bertambah satu, sedangkan jika ganjil maka Langkah akan berkurang satu. Setelah itu, terdapat pengecekan `if (k <= 0)` yang berfungsi sebagai mekanisme pengaman. Jika nilai `k` menjadi nol atau negatif akibat pengurangan sebelumnya, maka nilainya akan di-reset kembali ke `k_awal`.

Ketika perulangan `while` selesai, artinya hanya tersisa satu batu di dalam list. Program kemudian menampilkan batu yang tersisa tersebut melalui `cout << sll.get(0) << "\n"`; Fungsi `sll.clear()`; membersihkan seluruh sisa memori agar terhindar dari memory leak.

Misalkan ambil contoh output pertama, berikut adalah ilustrasi iterasi atau roses perulangan suatu rangkaian instruksi.



Penjelasan iterasi:

Iterasi pertama, banyak batu adalah 5 dengan $[1, 4, 3, 6, 2]$ dan langkah batu ialah 2. Lakukan rumus current_idx , $\text{current_idx} = (\text{current_idx} + k - 1) \% \text{sl1.size}$; atau $\text{current_idx} = (0 + 2 - 1) \% 5 = 1$. Pada indeks 1, terdapat angka 4 dan batu dihancurkan. Selanjutnya, karena 4 adalah genap maka langkah akan bertambah satu menjadi 3, Lakukan current_idx seperti yang sama maka akan mendapatkan nilai 3. Pada indeks ke-3 terdapat angka 2, lalu hancurkan,

Iterasi ketiga tersisa 3 angka, yakni $[1, 3, 6]$, karena tadi angka 2 adalah genap maka langkah akan bertambah menjadi 4. Hitung kembali current_idx maka akan mendapat angka 0. Di gambar sendiri $\text{current_idx} = (4 + 2 - 1) \% 5 = 1$. Maka angka 1 pada indeks pertama akan dihapus. Menyisakan angka 6 yang bertahan sampai akhir. Logika ini dinamakan logika Josephus.

SOAL 3: Implementasi Senarai Berantai Ganda Melingkar

General Technical Requirements:

Standard: C++23

Memory Management: Manual (new / delete)

Constraints: Strictly NO STL (<vector>, <list>, dll), NO Global Arrays.

Pada file `double_linked_list.h` yang diberikan, implementasikan struktur `DoubleLinkedList` beserta fungsi-fungsinya ke dalam file `double_linked_list.cpp`.

Jelaskan alur algoritma dari setiap metode yang diimplementasikan. Pastikan metode `clear()` berjalan dengan benar untuk menghindari memory leak (kebocoran memori). Lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi menggunakan Big-O Notation. Seluruh manajemen memori harus dialokasikan secara manual di dalam heap.

Jelaskan menggunakan alur bagaimana dan mengapa. Jelaskan kenapa algoritma yang kalian rancang itu valid dan yakin benar.

A. Source Code

Tabel 11. Source code `double_linked-list.cpp` Modul 3

```
1  #include "double_linked_list.h"
2  #include <iostream>
3  #include <stdexcept>
4
5  void DoubleLinkedList::init() {
6      head = nullptr;
7      tail = nullptr;
8      size = 0;
9  }
10
11 bool DoubleLinkedList::is_empty() {
12     return size == 0;
13 }
14
15 void DoubleLinkedList::add_front(int val) {
16     Node* newNode = new Node;
17     newNode->data = val;
18
19     if (is_empty()) {
20         head = tail = newNode;
21         newNode->next = head;
22         newNode->prev = tail;
23     } else {
24         newNode->next = head;
25         newNode->prev = tail;
26         head->prev = newNode;
27         tail->next = newNode;
28         head = newNode;
29     }
30     size++;
```

```

31 }
32
33 void DoubleLinkedList::add_back(int val) {
34     Node* newNode = new Node;
35     newNode->data = val;
36
37     if (is_empty()) {
38         head = tail = newNode;
39         newNode->next = head;
40         newNode->prev = tail;
41     } else {
42         newNode->next = head;
43         newNode->prev = tail;
44         tail->next = newNode;
45         head->prev = newNode;
46         tail = newNode;
47     }
48     size++;
49 }
50
51 void DoubleLinkedList::add_idx(int val, int idx) {
52     if (idx < 0 || idx > size) throw
std::out_of_range("Index tidak valid");
53     if (idx == 0) {
54         add_front(val);
55     } else if (idx == size) {
56         add_back(val);
57     } else {
58         Node* curr = head;
59         for (int i = 0; i < idx; i++) {
60             curr = curr->next;
61         }
62
63         Node* newNode = new Node;
64         newNode->data = val;
65
66         newNode->prev = curr->prev;
67         newNode->next = curr;
68
69         curr->prev->next = newNode;
70         curr->prev = newNode;
71         size++;
72     }
73 }
74
75 void DoubleLinkedList::delete_front() {
76     if (is_empty()) return;
77
78     if (size == 1) {
79         delete head;
80         head = tail = nullptr;
81     } else {

```

```

82         Node* temp = head;
83         head = head->next;
84         head->prev = tail;
85         tail->next = head;
86         delete temp;
87     }
88     size--;
89 }
90
91 void DoubleLinkedList::delete_back() {
92     if (is_empty()) return;
93
94     if (size == 1) {
95         delete tail;
96         head = tail = nullptr;
97     } else {
98         Node* temp = tail;
99         tail = tail->prev;
100        tail->next = head;
101        head->prev = tail;
102        delete temp;
103    }
104    size--;
105 }
106
107 void DoubleLinkedList::delete_idx(int idx) {
108     if (idx < 0 || idx >= size) throw
std::out_of_range("Index tidak valid");
109     if (idx == 0) {
110         delete_front();
111     } else if (idx == size - 1) {
112         delete_back();
113     } else {
114         Node* curr = head;
115         // Jalan ke target
116         for (int i = 0; i < idx; i++) {
117             curr = curr->next;
118         }
119         curr->prev->next = curr->next;
120         curr->next->prev = curr->prev;
121
122         delete curr;
123         size--;
124     }
125 }
126
127 void DoubleLinkedList::display() {
128     if (is_empty()) {
129         std::cout << "EMPTY\n";
130         return;
131     }
132     Node* curr = head;

```

```

133     do {
134         std::cout << curr->data;
135         curr = curr->next;
136     } while (curr != head);
137     std::cout << "\n";
138 }
139
140 char DoubleLinkedList::get(int idx) {
141     if (idx < 0 || idx >= size) throw
std::out_of_range("Index tidak valid");
142     Node* curr = head;
143     for (int i = 0; i < idx; i++) {
144         curr = curr->next;
145     }
146     return curr->data;
147 }
148
149 void DoubleLinkedList::set(char val, int idx) {
150     if (idx < 0 || idx >= size) throw
std::out_of_range("Index tidak valid");
151     Node* curr = head;
152     for (int i = 0; i < idx; i++) {
153         curr = curr->next;
154     }
155     curr->data = val;
156 }
157
158 void DoubleLinkedList::clear() {
159     while (!is_empty()) {
160         delete_front();
161     }
162 }

```

B. Pembahasan

1. Init()

berfungsi untuk menetapkan nilai awal struktur data. Kompleksitas waktu $O(1)$ karena hanya mengatur variabel dasar.

2. is_empty()

berfungsi untuk mengecek apakah list tersebut kosong dengan memvalidasi apakah variabel size bernilai 0. Kompleksitas Waktu $O(1)$, karena hanya mengecek satu variable.

3. add_front()

berfungsi untuk menambahkan data di posisi paling depan. Jika list masih kosong, node akan menjadi head sekaligus tail karena circular. Kompleksitas Waktu $O(1)$ karena tidak ada perulangan.

4. add_back()

berfungsi untuk menambahkan node di bagian belakang atau ekor list dengan cara membuat node baru dan mengisi nilainya lalu memperbarui variabel tail ke node paling belakang yang baru. Kompleksitas Waktu tetap $O(1)$ karena satu kondisi if tidak membatalkan status kompleksitas konstan $O(1)$ dimana Status $O(1)$ baru akan gugur kalau ada perulangan.

5. `add_idx()`

berfungsi untuk memuat posisi data. Setelah memastikan indeks tidak `out_of_range`, algoritma akan mendelegasikan ke `add_front` jika indeks 0, atau `add_back` jika indeks di ujung. Jika di tengah, program melakukan perulangan mencari letak node target (`curr`).

Kenapa valid & benar: Panah node baru diikat terlebih dahulu ke tetangga-tetangganya sebelum tali tetangga diputus dan diarahkan ke node baru. Ini menjaga keutuhan data. Kompleksitas waktu $O(N)$ karena butuh perulangan for.

6. `delete_front()`

berfungsi untuk menghapus node pertama guna menghindari memory leak. Jika hanya ada 1 node, list langsung di-reset ke `nullptr`. Jika banyak, head lama disimpan ke variabel `temp`. Setelah aman, `delete temp`; dipanggil untuk menghancurkan memori secara manual dari Heap. Kompleksitas Waktu $O(1)$.

7. `delete_back()`

berfungsi untuk menghapus node ekor guna menghindari memory leak tanpa perlu melakukan perulangan looping dari depan tail lama yang ditampung di `temp` dan dihancurkan ke dengan `delete temp`; Kompleksitas Waktu $O(1)$.

8. `delete_idx()`

berfungsi untuk menghapus node di untul menghindari memory leak. Setelah perulangan berjalan mencari letak dan menyambungkan node di kiri target `curr->prev->next` dengan node di kanan target `curr->next`. Setelah aman, barulah `delete curr;`. Kompleksitas Waktu $O(N)$ karena looping.

9. `display()`

berfungsi untuk mencetak semua isi node ke layar. Putaran akan berhenti ketika sudah menyentuh head. Kompleksitas Waktu $O(N)$.

10. `get()`

berfungsi untuk mengambil atau membaca nilai pada posisi indeks tertentu tanpa menghapusnya dari list. Program menggunakan perulangan `for` untuk menyusuri list langkah demi langkah. Setelah berjalan sebanyak `idx` dan `curr` berhenti tepat di node yang diminta, fungsi langsung mengembalikan nilai dari node tersebut ke program utama dengan command `return curr->data;`

11. `set()`

berfungsi untuk mengubah atau menimpa nilai data pada posisi (indeks) tertentu dengan nilai (`val`) yang baru. Jika indeksnya benar, penunjuk `curr` akan mulai dari `head->next` lalu menyusuri list menggunakan perulangan `for` sampai menemukan node target. Fungsi tidak mengembalikan nilai, melainkan langsung menimpa data lama di dalam node tersebut dengan data yang baru melalui command `curr->data = val;`

12. `clear()`

berfungsi s untuk menghapus semua elemen guna menghindari memory leak, sampai list benar-benar tak bersisa. Kompleksitas Waktu $O(N)$ karena menghapus node sebanyak jumlah data.

SOAL 4: Si Palui dan Deretan Angka

Si Palui mengamati fenomena aneh di atas sebuah gelanggang melingkar yang berisi N karakter. Terdapat dua proyektor anomali, Alpha (dimulai dari head, bergerak maju menggunakan next) dan Beta (dimulai dari tail, bergerak mundur menggunakan prev)

Palui melakukan observasi selama R ronde. Pada setiap ronde:

1. Proyektor Alpha dipaksa melompat maju sebanyak X langkah. Proyektor Beta dipaksa melompat mundur sebanyak Y langkah (simultan).
2. Tabrakan Singular: Jika Alpha dan Beta mendarat di titik atau karakter yang sama persis secara bersamaan, karakter tersebut akan hancur dan lenyap dari lingkaran dimensi. Jika ini terjadi, Alpha berpindah ke karakter next dari karakter yang hancur, dan Beta berpindah ke karakter prev dari karakter yang hancur.
3. Resonansi Bersebelahan (Adjacent): Jika Alpha dan Beta saling bersebelahan langsung, sifat kuantum dari kedua karakter tersebut saling bertukar (data antara kedua node tersebut di-swap).

Tentukan sisa karakter pada lingkaran dimensi setelah R ronde selesai dieksekusi

Batasan

- $1 \leq n \leq 100$
- $-1 \leq k \leq n$
- $-1000 \leq A_i \leq 1000$

Format Input

N R

$C_1 C_2 \dots C_N$

$X_1 Y_1$

$X_2 Y_2$

...

$X_R Y_R$

Keterangan: C_i adalah satu karakter char tunggal.

Format Output

Karakter yang tersisa, dicetak berurutan mulai dari head hingga ke elemen terakhir, tanpa spasi. Jika dimensi menjadi kosong, cetak EMPTY.

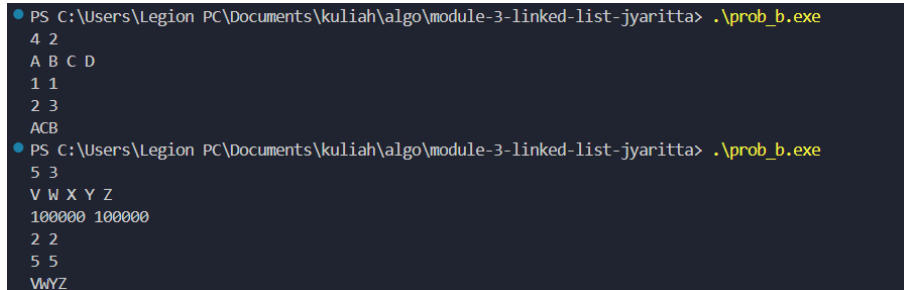
Input	Output
4 2 A B C D 1 1 2 3	ACB
5 3 V W X Y Z 100000 100000 2 2 5 5	VWYZ

A. Source Code

Tabel 12. Source code prob_b.cpp Modul 3

1	#include <iostream>
2	#include "queue.h"
3	
4	using namespace std;
5	
6	int main() {
7	int n, k;
8	if (!(cin >> n >> k)) return 0;
9	Queue window;
10	init(&window);
11	long long current_sum = 0;
12	int temp;
13	
14	for (int i = 0; i < k; i++) {
15	cin >> temp;
16	enqueue(&window, temp);
17	current_sum += temp;
18	}
19	cout << current_sum;
20	
21	for (int i = k; i < n; i++) {
22	cin >> temp;
23	current_sum -= front(&window);
24	~ dequeue(&window);
25	enqueue(&window, temp);
26	current_sum += temp;
27	cout << " " << current_sum;
28	}
29	cout << endl;
30	return 0;

B. Output Program



```

PS C:\Users\Legion PC\Documents\kuliah\algo\module-3-linked-list-jyaritta> .\prob_b.exe
4 2
A B C D
1 1
2 3
ACB
PS C:\Users\Legion PC\Documents\kuliah\algo\module-3-linked-list-jyaritta> .\prob_b.exe
5 3
V W X Y Z
100000 100000
2 2
5 5
VwYZ

```

Gambar 10. Screenshot Hasil Jawaban Soal 2 Modul 3

C. Pembahasan

Pada awal program, `cin >> N >> R` digunakan untuk membaca dua input dari pengguna, yaitu `N` sebagai jumlah total karakter dan `R` sebagai jumlah observasi. Pengecekan `if (!(cin >> n >> k)) return 0;` berfungsi untuk menghentikan program apabila input yang dimasukkan kosong atau tidak valid. Setelah itu, `arena.init();` untuk menyiapkan struktur Double Linked List.

Selanjutnya, program meminta input data karakter melalui perulangan `for (int i = 0; i < N; i++)`. Setiap karakter yang dibaca ditambahkan ke bagian belakang list menggunakan fungsi `arena.add_back(c)`. Program menetapkan posisi awal proyektor dimana `Node* alpha = arena.head;` dimulai dari depan dan `Node* beta = arena.tail;` yang dimulai dari belakang.

Selama jumlah `R` belum habis, seluruh logika pergerakan proyektor akan terus dieksekusi berulang kali di dalam `for (int i = 0; i < R; i++)`. Program membaca input jarak lompatan `X` dan `Y`. program memangkas jumlah lompatan menggunakan rumus `Modulo` `move_alpha = X % arena.size;` dan `move_beta = Y % arena.size;`. Alpha kemudian bergerak maju menggunakan `alpha = alpha->next`, sementara Beta bergerak mundur menggunakan `beta = beta->prev`.

Berikutnya, program mengecek atura. Jika `alpha == beta` atau berada di node yang sama, program akan mencatat pijakan proyektor selanjutnya. Jika tidak bertabrakan, program mengecek dengan kondisi `else if (alpha->next == beta || beta->next == alpha)`. Jika bersebelahan, muatan data dari kedua node tersebut saling ditukar (swap) menggunakan variabel `temp` tanpa menghancurkan node-nya.

Ketika perulangan ronde selesai, program akan menampilkan seluruh sisa karakter dari head hingga akhir menggunakan fungsi `arena.display()`. Terakhir, fungsi `arena.clear();` untuk menghancurkan sisa node di dalam memori agar terhindar dari memory leak.

MODUL 4: SORTING

Bubble Sort

A. Source Code

Tabel 13. Source code Bubble Sort Modul 4

```
1 void bubble_sort(std::vector<int>& data, Metrics& m) {
2     int n = data.size();
3     auto start = Clock::now();
4
5     for (int i=0; i < n-1; i++) {
6         for (int j=0; j < n-i- 1; j++) {
7             m.comparisons++;
8             if (data[j] > data[j + 1]) {
9                 std::swap(data[j], data[j + 1]);
10                m.swaps++;
11            }
12        }
13    }
14    m.time_ms = std::chrono::duration<double,
15    std::milli>(Clock::now() - start).count();
16 }
```

B. Output Program

>> Bubble Sort								
Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time(ms)
Bubble Sort	Random	100	4950	2564	0	0	0	0.016300
Bubble Sort	Sorted	100	4950	0	0	0	0	0.002800
Bubble Sort	Reverse Sorted	100	4950	4950	0	0	0	0.023100
Bubble Sort	Nearly Sorted	100	4950	296	0	0	0	0.004600
Bubble Sort	Many Duplicates	100	4950	2315	0	0	0	0.014900
Bubble Sort	Random	1000	499500	243190	0	0	0	1.178100
Bubble Sort	Sorted	1000	499500	0	0	0	0	0.277200
Bubble Sort	Reverse Sorted	1000	499500	499500	0	0	0	2.514700
Bubble Sort	Nearly Sorted	1000	499500	25778	0	0	0	0.357600
Bubble Sort	Many Duplicates	1000	499500	240808	0	0	0	1.408800
Bubble Sort	Random	10000	49995000	25118148	0	0	0	127.826200
Bubble Sort	Sorted	10000	49995000	0	0	0	0	21.484700
Bubble Sort	Reverse Sorted	10000	49995000	49995000	0	0	0	218.577200
Bubble Sort	Nearly Sorted	10000	49995000	3119352	0	0	0	35.654900
Bubble Sort	Many Duplicates	10000	49995000	25093086	0	0	0	128.146100

Gambar 11. Output Algoritma Bubble Sort Modul 4

C. Pembahasan

Dalam suatu perulangan pertama sebanyak $n - 1$ dimana n adalah size data tersebut, terdapat lagi perulangan didalamnya untuk membandingkan data pada indeks j atau data pertama yang akan dibandingkan dengan data pada indeks $j+1$ atau data di sebelahnya.

Jika $\text{data}[j]$ lebih besar dari $\text{data}[j+1]$ yang dibandingkan, maka tukar data tersebut. Selbihnya perulangan kedua ini akan terus terulang sampai $n - i - 1$ kali.

Setelah perulangan pertama fase i telah berakhir, data dengan ukuran terbesar sudah berada pada posisi paling belakang.



Dalam ketiga kasus, Worst case, Average case dan Best case. Kompleksitas waktunya adalah $O(n^2)$ pada ketiga kasus tersebut. Setiap jumlah data bertambah, maka jumlah operasi yang dibutuhkan program akan menjadi ganda juga secara eksponensial,

berarti bahwa waktu eksekusi suatu algoritma bertambah secara kuadratik seiring dengan bertambahnya ukuran input.

Algoritma ini termasuk stable karena tetap mempertahankan urutan relatif setelah proses pengurutan selesai. Algoritma ini termasuk algoritma In-Place Sorting karena tidak membutuhkan variabel atau rekursif tambahan.

Selection Sort

A. Source Code

Tabel 14. Source code Selection Sort Modul 4

```

1 void selection_sort(std::vector<int>& data, Metrics& m){
2     int n = data.size();
3     auto start = Clock::now();
4
5     for(int i = 0; i < n - 1; ++i){
6         int min_idx = i;
7         for(int j = i + 1; j < n; ++j){
8             m.comparisons++;
9             if(data[j]<data[min_idx]){
10                min_idx = j;
11            }
12        }
13        if (min_idx != i){
14            std::swap(data[i], data[min_idx]);
15            m.swaps++;
16        }
17        m.time_ms = std::chrono::duration<double,
18        std::milli>(Clock::now() - start).count();
19    }
19 }

```

B. Output Program

>> Selection Sort								
Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time(ms)
Selection Sort	Random	100	4950	95	0	0	0	0.010500
Selection Sort	Sorted	100	4950	0	0	0	0	0.010300
Selection Sort	Reverse Sorted	100	4950	50	0	0	0	0.010500
Selection Sort	Nearly Sorted	100	4950	4	0	0	0	0.010400
Selection Sort	Many Duplicates	100	4950	89	0	0	0	0.010500
Selection Sort	Random	1000	499500	995	0	0	0	0.897800
Selection Sort	Sorted	1000	499500	0	0	0	0	0.869300
Selection Sort	Reverse Sorted	1000	499500	500	0	0	0	0.854600
Selection Sort	Nearly Sorted	1000	499500	50	0	0	0	0.869900
Selection Sort	Many Duplicates	1000	499500	983	0	0	0	0.871500
Selection Sort	Random	10000	49995000	9992	0	0	0	83.594000
Selection Sort	Sorted	10000	49995000	0	0	0	0	84.382600
Selection Sort	Reverse Sorted	10000	49995000	5000	0	0	0	83.730600
Selection Sort	Nearly Sorted	10000	49995000	500	0	0	0	88.024900
Selection Sort	Many Duplicates	10000	49995000	9981	0	0	0	86.022000

Gambar 12. Output Algoritma Selection Sort Modul 4

C. Pembahasan

Dalam suatu perulangan pertama sebanyak $n - 1$ dimana n adalah size data tersebut dan membuatnya menjadi indeks nilai minimum sementara $\text{min_idx} = i$.

Di dalamnya, terdapat lagi perulangan kedua yang berjalan sebanyak $n - i - 1$ kali untuk mencari data paling kecil dengan cara membandingkan data tersebut dengan $\text{data}[j]$. Jika $\text{data}[j]$ lebih kecil maka min_idx akan diubah nilainya menjadi j . Dimana ini menyimpan indeks suatu data terkecil.

Setelah perulangan kedua selesai, program akan mengecek apakah min_idx sudah berubah dengan $\text{min_idx} \neq i$. Jika iya, maka dilakukan satu kali penukaran data dengan std::swap . Jika tidak ada perubahan, posisi data dibiarkan. Setelah seluruh i fase perulangan pertama berakhir, data dengan ukuran terkecil akan berada di posisi paling depan.

Dalam kasus Best, Average dan Worst nilai Comparisons konsisiten pada jenis-jenis data yang dimana kompleksitas waktunya adalah $O(n^2)$, waktu eksekusi algoritma ini bertambah secara kuadratik seiring dengan bertambahnya ukuran data.

Algoritma Selection Sort termasuk Unstable karena dapat mengubah urutan relatif data yang memiliki nilai yang sama. Algoritma ini membutuhkan satu alokasi memori tambahan untuk menyimpan variabel indeks sementara.

Insertion Sort

A. Source Code

Tabel 15. Source code Insertion Sort Modul 4

```

1 void insertion_sort(std::vector<int>& data, Metrics& m)
2 {
3     int n = data.size();
4     auto start = Clock::now();
5
6     for (int i = 1; i < n; ++i) {
7         int key = data[i];
8         int j = i - 1;
9         m.comparisons++;
10
11         while (j >= 0 && data[j] > key) {
12             data[j + 1] = data[j];
13             m.shifts++;
14             j = j - 1;
15             m.comparisons++;
16         }
17         data[j + 1] = key;
18     }
19     m.time_ms = std::chrono::duration<double,
20 std::milli>(Clock::now() - start).count();
21 }

```

B. Output Program

>> Insertion Sort								
Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time(ms)

Insertion Sort	Random	100	2663	0	2564	0	0	0.002400
Insertion Sort	Sorted	100	99	0	0	0	0	0.000100
Insertion Sort	Reverse Sorted	100	5049	0	4950	0	0	0.006200
Insertion Sort	Nearly Sorted	100	395	0	296	0	0	0.000700
Insertion Sort	Many Duplicates	100	2414	0	2315	0	0	0.002100
Insertion Sort	Random	1000	244189	0	243190	0	0	0.109500
Insertion Sort	Sorted	1000	999	0	0	0	0	0.000700
Insertion Sort	Reverse Sorted	1000	500499	0	499500	0	0	0.240300
Insertion Sort	Nearly Sorted	1000	26777	0	25778	0	0	0.031100
Insertion Sort	Many Duplicates	1000	241807	0	240808	0	0	0.119900
Insertion Sort	Random	10000	25128147	0	25118148	0	0	11.492300
Insertion Sort	Sorted	10000	9999	0	0	0	0	0.005800
Insertion Sort	Reverse Sorted	10000	50004999	0	49995000	0	0	22.017600
Insertion Sort	Nearly Sorted	10000	3129351	0	3119352	0	0	1.395600
Insertion Sort	Many Duplicates	10000	25103085	0	25093086	0	0	10.702500
>> Merge Sort								

Gambar 13. Output Algoritma Insertion Sort Modul 4

C. Pembahasan

Algoritma ini melakukan perulangan dengan indeks $i = 1$ sampai i tidak melebihi n dimana n adalah size data tersebut. Program akan mengambil elemen pada indeks saat ini menjadi $key = data[i]$ sebagai pembanding nantinya dan $j = i - 1$ sebagai batas indeks.

Terdapat perulangan kedua dengan struktur `while` untuk mengecek data ke kiri selama syarat masih $j \geq 0$ dan nilai $data[j] > key$. Jika kondisi perulangan `while` terpenuhi, maka data indeks j akan digeser satu posisi ke kanan dengan $data[j + 1] = data[j]$. Nilai j akan terus dikurangi satu setiap bergeser. Di luar loop, mengubah key $data[j + 1] = key$ untuk untuk menaruh elemen key ke indeks yang sudah benar.

Setelah seluruh fase i perulangan pertama berakhir, seluruh data sukses terurut dari elemen terkecil di depan hingga terbesar di belakang.

Kompleksitas waktu pada Best Case adalah $O(n)$ karena tidak ada syarat yang memenuhi loop `while` karena sudah terurut. Sedangkan Average dan Worst, nilai Comparisons dan Shifts tinggi dan waktu eksekusi bertambah secara eksponensial seiring bertambahnya ukuran data.

Algoritma ini stabil karena ketika ada data dengan nilai sama, data tersebut akan tetap digeser di sebelah kanan data dengan nilai yang sama yang lama, sehingga urutan relatif asli tidak berubah. Algoritma ini in-place karena hanya memerlukan satu memori tambahan sementara tanpa memerlukan memerlukan panggilan rekursif.

Merge Sort

A. Source Code

Tabel 16. Source code Merge Sort Modul 4

```
1 void merge(std::vector<int>& data, int left, int mid, int
  right, Metrics& m){
2     int n1 = mid - left + 1;
3     int n2 = right - mid;
4
5     std::vector<int> L(n1), R(n2);
6
7     for (int i = 0; i < n1; i++) L[i] = data[left + i];
8     for (int j = 0; j < n2; j++) R[j] = data[mid + 1 +
  j];
9
10    int i = 0, j = 0, k = left;
11
12    while (i < n1 && j < n2) {
13        m.comparisons++;
14        if (L[i] <= R[j]) {
15            data[k] = L[i];
16            i++;
17        } else {
18            data[k] = R[j];
19            j++;
20        }
21        k++;
22    }
23
24    while (i < n1){ data[k] = L[i]; i++; k++; }
25    while (j < n2){ data[k] = R[j]; j++; k++; }
26 }
27
28 void divide(std::vector<int>& data, int left, int right,
  Metrics& m) {
29     m.recursive_calls++;
30     if (left >= right) return;
31
32     int mid = left + (right - left) / 2;
33     divide(data, left, mid, m);
34     divide(data, mid + 1, right, m);
35     merge(data, left, mid, right, m);
36 }
37
38 void merge_sort(std::vector<int>& data, Metrics& m) {
39     int n = data.size();
40     if (n <= 1) return;
41
42     auto start = Clock::now();
43     divide(data, 0, n - 1, m);
```

```

44     m.time_ms = std::chrono::duration<double,
std::milli>(Clock::now() - start).count();
45 }

```

B. Output Program

>> Merge Sort								
Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time(ms)
Merge Sort	Random	100	535	0	0	199	0	0.012800
Merge Sort	Sorted	100	356	0	0	199	0	0.019600
Merge Sort	Reverse Sorted	100	316	0	0	199	0	0.012800
Merge Sort	Nearly Sorted	100	453	0	0	199	0	0.009700
Merge Sort	Many Duplicates	100	536	0	0	199	0	0.017500
Merge Sort	Random	1000	8690	0	0	1999	0	0.105200
Merge Sort	Sorted	1000	5044	0	0	1999	0	0.162600
Merge Sort	Reverse Sorted	1000	4932	0	0	1999	0	0.131300
Merge Sort	Nearly Sorted	1000	7628	0	0	1999	0	0.135800
Merge Sort	Many Duplicates	1000	8679	0	0	1999	0	0.138300
Merge Sort	Random	10000	120465	0	0	19999	0	1.578200
Merge Sort	Sorted	10000	69008	0	0	19999	0	1.615900
Merge Sort	Reverse Sorted	10000	64608	0	0	19999	0	1.379700
Merge Sort	Nearly Sorted	10000	111524	0	0	19999	0	1.439600
Merge Sort	Many Duplicates	10000	120461	0	0	19999	0	1.706000

Gambar 14. Output Algoritma Merge Sort Modul 4

C. Pembahasan

Pada baris [1 sampai 4] prob2.cpp, syntax `#include <iostream>` untuk mengimpor input/output dan memanggil header "queue.h" agar program utama mengenali fungsi yang telah dibuat.

Dalam algoritma Merge Sort, terdapat tambahan 2 rekursif di source code. Masing-masing kegunaannya sebagai berikut:

Di dalam fungsi merge, membuat dua buah vektor sementara yaitu L dan R untuk menyalin data kiri dan kanan. Terdapat perulangan `while i < n1 && j < n2` untuk membandingkan data dua vektor tersebut. Jika `L[i] <= R[j]`, maka data dari vektor kiri akan dimasukkan ke vektor utama `data[k] = L[i]`, jika tidak maka data vektor kanan yang akan dimasukkan ke `data[k] = R[j]`. Sisa-sisa data yang belum dimasukkan akan dilanjutkan oleh dua loop `while` selanjutnya. Setelah seluruh fase perulangan rekursif merge berakhir, seluruh data akan terurut dari yang terkecil.

Dalam fungsi divide yang menerima nilai `left` dan `right` untuk membelah data menjadi dua bagian berdasarkan nilai tengahnya `mid = left + (right - left) / 2`. Fungsi ini memanggil dirinya sendiri `divide(data, left, mid, m)` untuk membelah diri lagi bagian kiri dan `divide(data, mid + 1, right, m)` untuk bagian kanan. Perulangan rekursif ini akan terus berjalan dan berhenti ketika `left >= right` yang berarti data sudah terurut sampai tersisa satu elemen. Setelah itu, program memanggil fungsi merge untuk menggabungkan kembali potongan-potongan atau array dari data tersebut.

Fungsi `merge_sort` sendiri hanya mempersiapkan batas indeks data, 0, n-1, m dan mencatat waktu eksekusi. Dalam ketiga kasus, kompleksitas waktunya adalah $O(n \log n)$ karena nilai Comparisons dan Rec.Calls stabil pada jenis-jenis data karena pembelahan data yang dibentuk akan selalu sama alurnya. Algoritma ini stabil karena penggunaan tanda `<=`, sehingga ketika ada data bernilai sama, nilai relatif asli tidak berubah. Algoritma ini tidak termasuk in-place karena membutuhkan rekursif tambahan.

Quick Sort

A. Source Code

Tabel 17. Source code Quick Sort Modul 4

1	void partition(std::vector<int>& data, int low, int high, Metrics& m) {
2	m.recursive_calls++;
3	
4	if (low >= high) return;
5	
6	int pivot = data[high];
7	int i = (low - 1);
8	
9	for (int j = low; j <= high - 1; j++){
10	m.comparisons++;
11	if (data[j] <= pivot){
12	i++;
13	std::swap(data[i], data[j]);
14	m.swaps++;
15	}
16	}
17	
18	std::swap(data[i + 1], data[high]);
19	m.swaps++;
20	
21	int pi = i + 1;
22	
23	partition(data, low, pi - 1, m);
24	partition(data, pi + 1, high, m);
25	}
26	
27	void quick_sort(std::vector<int>& data, Metrics& m) {
28	int n = data.size();
29	if (n <= 1) return;
30	
31	auto start = Clock::now();
33	partition(data, 0, n - 1, m);
34	m.time_ms = std::chrono::duration<double,
35	std::milli>(Clock::now() - start).count();
36	}

B. Output Program

```
>> Quick Sort
```

Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time(ms)
Quick Sort	Random	100	676	396	0	131	0	0.003300
Quick Sort	Sorted	100	4950	5049	0	199	0	0.004000
Quick Sort	Reverse Sorted	100	4950	2549	0	199	0	0.003500
Quick Sort	Nearly Sorted	100	2638	2607	0	191	0	0.003100
Quick Sort	Many Duplicates	100	832	707	0	181	0	0.002200
Quick Sort	Random	1000	10530	6168	0	1337	0	0.033400
Quick Sort	Sorted	1000	499500	500499	0	1999	0	0.345700
Quick Sort	Reverse Sorted	1000	499500	250499	0	1999	0	0.198000
Quick Sort	Nearly Sorted	1000	35189	25081	0	1611	0	0.045100
Quick Sort	Many Duplicates	1000	12588	9665	0	1801	0	0.037500
Quick Sort	Random	10000	149797	77759	0	13347	0	0.388500
Quick Sort	Sorted	10000	49995000	50004999	0	19999	0	25.188100
Quick Sort	Reverse Sorted	10000	49995000	25004999	0	19999	0	17.405100
Quick Sort	Nearly Sorted	10000	622473	458547	0	16511	0	0.449700
Quick Sort	Many Duplicates	10000	170629	113205	0	18007	0	0.356900

Gambar 15. Output Algoritma Quick Sort Modul 4

C. Pembahasan

Algoritma ini melakukan perulangan dengan indeks $i = 1$ sampai i tidak melebihi n dimana n adalah size data tersebut. Program akan mengambil elemen pada indeks saat ini menjadi $key = data[i]$ sebagai pembanding nantinya dan $j = i - 1$ sebagai batas indeks.

Di dalam fungsi `partition`, Jika syarat $low \geq high$ terpenuhi, fungsi akan berhenti karena sub-array tidak bisa dibagi lagi. Jika tidak, program akan menentukan data terakhir sebagai pembatas $pivot = data[high]$ dan indeks $i = low - 1$.

Terdapat perulangan kedua `for` dimana melakukan perulangan dari $j = low$ sampai $high - 1$ untuk mengecek data. Di dalam loop, terdapat perbandingan jika $data[j] \leq pivot$, maka $i++$ dan melakukan penukaran posisi menggunakan `std::swap(data[i], data[j])`.

Setelah perulangan selesai, program menempatkan pivot dengan menukarnya via `std::swap(data[i + 1], data[high])`. Langkah terakhir pada fungsi `partition` ini adalah memanggil dirinya dua kali dengan melalui `partition(data, low, pi - 1, m)` untuk memproses sisi kiri pivot, dan `partition(data, pi + 1, high, m)` untuk memproses sisi kanan pivot. Setelah seluruh fase perulangan rekursif berakhir, data akan terurut dari elemen terkecil.

Dalam fungsi utama `quick_sort`, pertama, menyimpan ukuran data menggunakan `int n = data.size()`. Jika data hanya berisi satu elemen atau kosong $n \leq 1$, maka program akan berhenti. Jika data lebih dari satu elemen, memulai waktu awal eksekusi. Lalu, memanggil `partition(data, 0, n - 1, m)` untuk memulai proses pembagian dan pengurutan data indeks paling awal 0 dan paling akhir $n - 1$.

Berdasarkan output performa algoritma Quick Sort ini menunjukkan perbedaan best dan average case dengan worst case. Dimana best dan average case $O(n \log n)$ cenderung stabil berdasarkan waktu eksekusinya. Worst Case termasuk $O(n^2)$ karena jumlah operasi perbandingan dan waktu eksekusinya meningkat secara eksponensial.

Algoritma Quick Sort Unstable karena proses pemindahan data bisa melompati data dengan nilai sama sehingga dapat merusak urutan relatif. Algoritma ini termasuk In-Place Sorting karena masih menggunakan `std::swap`.

Radix Sort

A. Source Code

Tabel 18 Source code Radix Sort Modul 4

```
1  int getMax(const std::vector<int>& data, int n, Metrics&
   m) {
2      int mx = data[0];
3      m.array_accesses++;
4      for (int i = 1; i < n; i++) {
5          if (data[i] > mx) {
6              mx = data[i];
7          }
8      }
9      return mx;
10 }
11
12 void countSort(std::vector<int>& data, int n, int exp,
   Metrics& m) {
13     std::vector<int> output(n);
14     int count[10] = { 0 };
15
16     for (int i = 0; i < n; i++) {
17         count[(data[i] / exp) % 10]++;
18         m.array_accesses++;
19     }
20
21     for (int i = 1; i < 10; i++) {
22         count[i] += count[i - 1];
23         m.array_accesses += 2;
24     }
25
26     for (int i = n - 1; i >= 0; i--) {
27         output[count[(data[i] / exp) % 10] - 1] = data[i];
28         m.array_accesses += 2;
29
30         count[(data[i] / exp) % 10]--;
31         m.array_accesses++;
32     }
33
34     for (int i = 0; i < n; i++) {
35         data[i] = output[i];
36         m.array_accesses++;
37     }
38 }
39
40 void radix_sort(std::vector<int>& data, Metrics& m) {
41     int n = data.size();
42     if (n <= 1) return;
```

```

42     auto start = Clock::now();
43
44     int max_val = getMax(data, n, m);
45
46     for (int exp = 1; max_val / exp > 0; exp *= 10) {
47         countSort(data, n, exp, m);
48     }
49     m.time_ms = std::chrono::duration<double,
std::milli>(Clock::now() - start).count();
50 }

```

B. Output Program

>> Quick Sort								
Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time(ms)
Quick Sort	Random	100	676	396	0	131	0	0.003300
Quick Sort	Sorted	100	4950	5049	0	199	0	0.004000
Quick Sort	Reverse Sorted	100	4950	2549	0	199	0	0.003500
Quick Sort	Nearly Sorted	100	2638	2607	0	191	0	0.003100
Quick Sort	Many Duplicates	100	832	707	0	181	0	0.002200
Quick Sort	Random	1000	10530	6168	0	1337	0	0.033400
Quick Sort	Sorted	1000	499500	500499	0	1999	0	0.345700
Quick Sort	Reverse Sorted	1000	499500	250499	0	1999	0	0.198000
Quick Sort	Nearly Sorted	1000	35189	25081	0	1611	0	0.045100
Quick Sort	Many Duplicates	1000	12588	9665	0	1801	0	0.037500
Quick Sort	Random	10000	149797	77759	0	13347	0	0.388500
Quick Sort	Sorted	10000	49995000	50004999	0	19999	0	25.188100
Quick Sort	Reverse Sorted	10000	49995000	25004999	0	19999	0	17.405100
Quick Sort	Nearly Sorted	10000	622473	458547	0	16511	0	0.449700
Quick Sort	Many Duplicates	10000	170629	113205	0	18007	0	0.356900

Gambar 16. Output Algoritma Radix Sort Modul 4

C. Pembahasan

Di dalam fungsi `getMax`, menetapkan elemen pertama sebagai nilai maksimum `mx = data[0]`. Terdapat perulangan untuk memeriksa seluruh data di dalam array. Di perulangan tersebut terdapat kondisi, jika `data[i] > mx` maka nilai `mx` akan diganti menjadi data indeks `i` yang lebih besar daripada nilai `mx` menjadi `data[i]`. Setelah itu, mengembalikan nilai `mx`.

Di dalam fungsi `countSort`, `Output(n)` berfungsi untuk menyimpan hasil urutan sementara dan array `count[10] = { 0 }` untuk menghitung berapa kali digit 0-9 muncul. Terdapat empat perulangan `for` yang berjalan secara berurutan:

1. Perulangan pertama berjalan sebanyak `i=0` sampai `i<n`, menghitung jumlah kemunculan setiap digit dari satu elemen dari data tiap iterasinya, menggunakan rumus $(data[i] / exp) \% 10$. Sebagai contoh 184 akan menghasilkan satuan 4 lalu `count[4]` bertambah satu. Proses ini terus dilakukan sampai seluruh elemen selesai diperiksa.
2. Setelah seluruh digit dihitung, perulangan kedua dari `i=1` hingga `i<10` mengubah array `count` menjadi posisi kumulatif agar program tahu posisi akhir setiap digit array dengan `count[i] += count[i - 1]`.

4	2	3	1	0	10	2	0	2	1
0	1	2	3	4	5	6	7	8	9

4	6	3	1						
0	1	2	3	4	5	6	7	8	9

4	6	9	1	0	10	2	0	2	1
0	1	2	3	4	5	6	7	8	9

Hasil Akhir :

4	6	9	10	10	20	22	22	24	25
0	1	2	3	4	5	6	7	8	9

- Selanjutnya, perulangan ketiga berulang sebanyak $i = n-1$ sampai $i \geq 0$, dimana tahap ini mengecek setiap elemen dari bagian paling kanan ke kiri dan memasukkannya ke posisi array output dengan $\text{count}[(\text{data}[i] / \text{exp}) \% 10] - 1$. Setelah elemen ditempatkan, nilai count untuk digit tersebut dikurangi satu agar posisi berikutnya berpindah ke indeks sebelumnya.
- perulangan terakhir dari $i=0$ hingga $i < n$ menyalin seluruh array output dan mengembalikannya ke array data.

Dalam fungsi `radix_sort`, memanggil fungsi `getMax(data, n, m)` untuk mencari nilai tertinggi dan mengetahui digit maksimumnya. Setelah itu, program melakukan perulangan `for (int exp = 1; max_val / exp > 0; exp *= 10)` yang akan berjalan terus berdasarkan banyak digit nilai tertinggi tersebut, nilai max tersebut akan dibagi oleh `exp`, untuk perulangan pertama akan mengurutkan satuan, lalu pada perulangan kedua akan mengurutkan puluhan. `exp` akan bertambah `exp*=10` sampai syarat `max_val / exp > 0` tidak lagi terpenuhi.

Di setiap iterasinya, memanggil `countSort(data, n, exp, m)` untuk merapikan data berdasarkan digitnya. misal satuan, puluhan, ratusan atau ribuan dan seterusnya. Setelah fase perulangan berakhir, seluruh data akan terurut dari yang terkecil.

Berdasarkan hasil output, Best, Average, dan Worst Case adalah $O(n*d)$ karena cara kerjanya yang murni berupa perulangan di dalam perulangan. Dari output, terlihat kompleksitas waktu yang sama rata di ketiga kasus konsisten dan stabil

Algoritma Radix Sort termasuk Stable karena data dipindahkan secara mundur dari belakang sehingga urutan relatif asli tidak berubah. Algoritma ini tidak termasuk In-Place Sorting karena membutuhkan alokasi memori tambahan.

KESIMPULAN

Berdasarkan output hasil, berikut adalah urutan algoritma dari yang paling cepat:

1. Radix Sort
2. Quick Sort
3. Merge Sort
4. Insertion Sort
5. Selection Sort
6. Bubble Sort

Radix Sort adalah algoritma tercepat pertama dengan waktu eksekusi rata-rata sekitar 0.16 ms – 0.17 ms ketika $N = 10.000$. Waktu eksekusi percobaan semuanya di bawah 1.0 ms pada seluruh pengujian. Sedangkan bubble sort paling lambat terakhir karena harus membandingkan dan menukar elemen terlebih dahulu. Ini membuat bubble sort tidak efisien untuk data besar.

MODUL 5: SEARCHING

LINEAR SEARCH

A. Source Code

Tabel 19. Source code Linear Search Modul 5

```
1 void linear_search(const std::vector<int>& data, int
  target, Metrics& m) {
2     m.reset();
3     auto start = Clock::now();
4     m.n = data.size();
5
6     for (int i = 0; i < data.size(); i++) {
7         m.comparisons++;
8         if (data[i] == target) {
9             m.found_index = i;
10            break;
11        }
12    }
13    m.time_us = std::chrono::duration<double,
std::micro>(Clock::now() - start).count();
14 }
```

B. Output Program

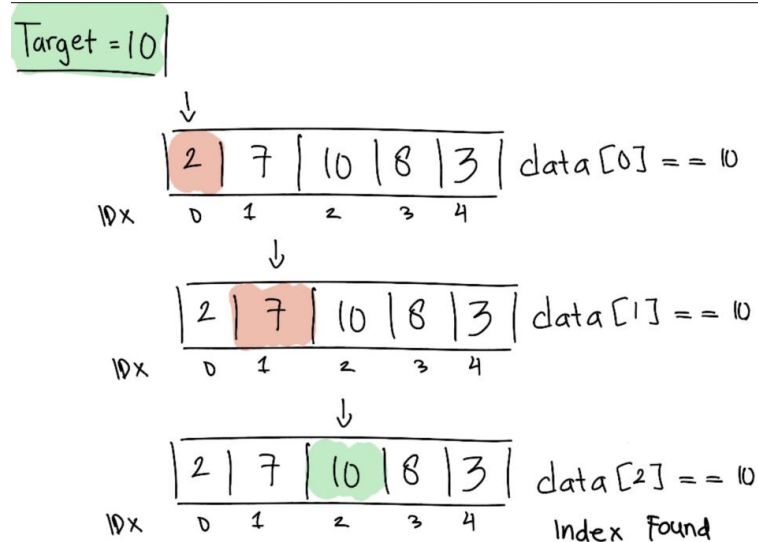
Algorithm	Case	N	Comparisons	Found Index	Time(us)
Linear Search	Best	1000	1	0	0.100
Linear Search	Average	1000	375	374	1.700
Linear Search	Worst	1000	1000	Not Found	4.500
Linear Search	Best	10000	1	0	0.100
Linear Search	Average	10000	3746	3745	16.200
Linear Search	Worst	10000	10000	Not Found	42.700
Linear Search	Best	100000	1	0	0.000
Linear Search	Average	100000	37455	37454	180.800
Linear Search	Worst	100000	100000	Not Found	465.700
Linear Search	Best	1000000	1	0	0.200
Linear Search	Average	1000000	374541	374540	1983.600
Linear Search	Worst	1000000	1000000	Not Found	4595.500

Gambar 17. Output Algoritma Linear Search Modul 5

C. Pembahasan

Pertama-tama, algoritma ini melakukan perulangan dengan indeks $i = 0$ sampai $i < \text{data.size}()$. Terdapat pengecekan suatu kondisi dengan `if`, apakah elemen pada indeks saat

ini sama dengan target dengan `data[i] == target`. Program ini akan terus melakukan perulangan sampai kondisi terpenuhi dan data ditemukan, program menyimpan indeks tersebut ke dalam `m_found_index = i` dan menghentikan program dengan `break` untuk menghentikan perulangan tanpa harus menunggu kondisi loop terpenuhi. Jika perulangan selesai dan target tidak ditemukan, program akan menetapkan `m_found_index = -1`.



Linear Search tidak membutuhkan data terurut karena algoritma ini berjalan dengan membandingkan satu persatu tiap elemen data dengan target atau bersifat iteratif karena menggunakan perulangan for dari indeks awal sampai akhir meskipun bisa diterapkan secara rekursif jika mau. Untuk kompleksitas waktu sendiri, Best case memiliki $\Omega(1)$ karena loop akan berjalan sekali karena data berada pada indeks paling pertama. Average dan Worst memiliki kompleksitas waktu $O(n)$. Untuk Average performa kodenya sangat bergantung pada seberapa besar ukuran data. Lalu, Worst dimana target berada di paling akhir atau not found, maka loop harus memeriksa seluruh n data sebagaimana di output comparasions bernilai sama dengan n .

BINARY SEARCH

A. Source Code

Tabel 20. Source code Binary Search Modul 5

```

1 void binary_search(const std::vector<int>& data, int
target, Metrics& m) {
2     m.reset();
3     m.n = data.size();
4     auto start = Clock::now();
5
6     int low = 0;
7     int high = data.size() - 1;
8     while (low <= high) {
9         int mid = low + (high - low) / 2;
10
11         m.comparisons++;
12         if (data[mid] == target) {
13             m.found_index = mid;
14             break;
15         }
16         if (data[mid] < target)
17             low = mid + 1;
18         else
19             high = mid - 1;
20     }
21     m.time_us = std::chrono::duration<double,
22 std::micro>(Clock::now() - start).count();
23 }

```

B. Output Program

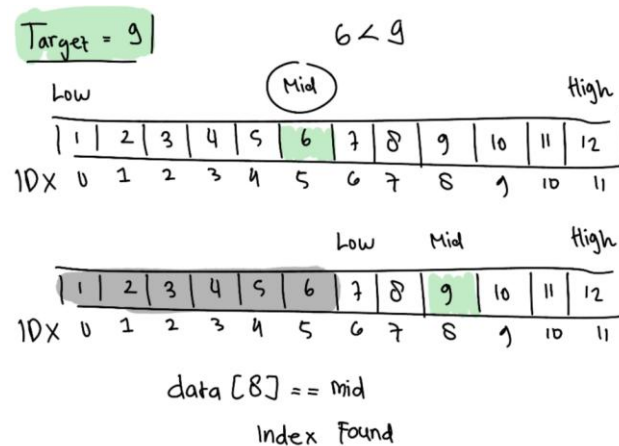
>> Binary Search					
Algorithm	Case	N	Comparisons	Found Index	Time(us)
Binary Search	Best	1000	9	500	0.400
Binary Search	Average	1000	3	374	0.300
Binary Search	Worst	1000	10	Not Found	0.400
Binary Search	Best	10000	13	5000	0.200
Binary Search	Average	10000	13	3745	0.300
Binary Search	Worst	10000	14	Not Found	0.500
Binary Search	Best	100000	16	50000	0.200
Binary Search	Average	100000	16	37454	0.700
Binary Search	Worst	100000	17	Not Found	0.600
Binary Search	Best	1000000	19	500000	1.200
Binary Search	Average	1000000	15	374540	1.200
Binary Search	Worst	1000000	20	Not Found	1.000

Gambar 18. Output Algoritma Binart Search Modul 5

C. Pembahasan

Algoritma ini bekerja dengan membagi ukuran data menjadi dua bagian secara terus-menerus hingga data yang dicari ditemukan. Pertama-tama, menetapkan `low = 0` dan batas akhir dengan `high = data.size() - 1`. Dalam perulangan pertama dengan `while` yang berjalan selama syarat kondisi `low <= high` terpenuhi. Program mencari indeks nilai tengah dengan rumus `mid = low + (high - low) / 2`. Selanjutnya terdapat pengecekan dengan `if`. Dimana, `if` pertama memeriksa apakah data di tengah sama dengan target. Jika kondisi ini terpenuhi, program menyimpan indeks tersebut ke dalam `m_found_index = mid` dan langsung menghentikan fungsi dengan menghentikan perulangan, tanpa harus menunggu kondisi loop selesai..

Jika tidak sama, `if` kedua mengecek jika `data[mid] < target`, maka `low` akan diperbarui ke `mid + 1`, atau secara mudahnya data dibagi menjadi dua dan menaruh nilai `low` di bagian yang sesuai seperti `low` akan berada tetap tetapi `high` akan berubah ke `mid - 1` ketika target lebih besar dari nilai `mid`, begitu pula sebaliknya. Lebih mudahnya, seperti gambaran berikut:



Binary Search perlu data yang sudah terurut karena menjadi dua bagian data berpegang dengan kurang dari atau lebih besarnya suatu target. Algoritma ini bersifat iteratif karena menggunakan perulangan `while` meskipun bisa diterapkan secara rekursif jika mau. Untuk kompleksitas waktu sendiri, Best Case memiliki $O(1)$ karena target berada tepat di nilai `mid` pertama dan hanya melakukan satu perulangan dan perbandingan sekali. Sedangkan untuk Average dan Worst memiliki kompleksitas waktu $O(\log n)$. Karena cara kerja algoritma ini membelah ruang pencarian menjadi setengah setiap iterasi. Untuk case Average, target diacak dan bisa saja ditemukan di tengah proses pembelahan, sementara Worst Case berada di ujung belahan atau not found sebagaimana dibuktikan pada output Worst-Case dimana jumlah komparasi maksimalnya bernilai mendekati dengan uji coba rumus kompleksitas.

Kompleksitas Waktu	Hasil Logaritma	Jumlah Comparassion
$\log_2(1000)$	9.96578428466	9
$\log_2(10000)$	13.2877123795	13
$\log_2(100000)$	16.6096404744	16
$\log_2(1000000)$	19.9315685693	19

--	--	--

MODUL 6: GRAPH AND TREE

SOAL 1: Konversi Adjacency List ke Adjacency Matrix

Time Limit: 1.0 s
Memory Limit: 64 MB

Deskripsi Soal

Diberikan sebuah **directed weighted graph** yang terdiri dari N vertex dan M edge. Setiap vertex memiliki label berupa satu karakter unik. Graph diberikan dalam bentuk daftar edge.

Setiap edge ditulis sebagai U, V, W , yang berarti terdapat edge berarah dari vertex U menuju vertex V dengan bobot W . Karena graph bersifat berarah, edge U, V, W tidak otomatis berarti terdapat edge dari V menuju U .

Tugas Anda adalah mengubah representasi graph tersebut menjadi **adjacency matrix**. Jika terdapat edge dari vertex ke- i menuju vertex ke- j , maka nilai matrix pada baris i dan kolom j adalah bobot edge tersebut. Jika tidak terdapat edge, nilainya 0.

Format Input

Input diberikan dalam format berikut.

```
N
V1 V2 ... VN
M
U1 V1 W1 U2 V2 W2
...
UM VM WM
```

Keterangan.

Keterangan:

- N adalah jumlah vertex.
- V_i adalah label vertex ke- i .
- M adalah jumlah edge.
- U_i dan V_i adalah vertex asal dan vertex tujuan pada edge ke- i .
- W_i adalah bobot edge ke- i .

Format Output

Cetak adjacency matrix dengan format berikut.

```
Adjacency Matrix:
V1 V2 ... VN
```

```

V1 a11 a12 ... a1N
V2 a21 a22 ... a2N

...

VN aN1 aN2 ... aNN

```

Nilai a_{ij} adalah bobot edge dari vertex V_i menuju vertex V_j . Jika edge tersebut tidak ada, nilai a_{ij} adalah 0.

Batasan

- $2 \leq N \leq 10$
- $0 \leq M \leq N * (N - 1)$
- $1 \leq W_i \leq 1000$
- Label vertex berupa karakter alfabet kapital A sampai Z.
- Semua label vertex dijamin unik.
- Graph tidak memiliki **self-loop**.
- Graph tidak memiliki edge duplikat dengan arah yang sama.
- Graph bersifat berarah dan berbobot.

Contoh Kasus

Input	Output
5 A B C D E 6 A B 4 A C 2 B D 7 C B 1 C E 6 D C 3	Adjacency Matrix: A B C D E A 0 4 2 0 0 B 0 0 0 7 0 C 0 1 0 0 6 D 0 0 3 0 0 E 0 0 0 0 0

Penjelasan Contoh

Edge A B 4 menghasilkan nilai 4 pada baris A kolom B. Karena graph berarah, nilai pada baris B kolom A tetap 0 selama tidak ada edge dari B menuju A.

Edge D C 3 menghasilkan nilai 3 pada baris D kolom C. Vertex E tidak memiliki edge keluar, sehingga seluruh nilai pada baris E adalah 0.

A. Source Code

Tabel 21. Source code prob1.cpp Modul 6

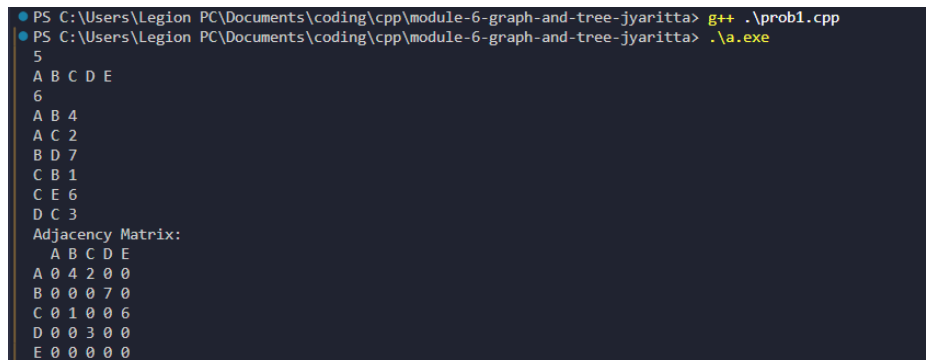
1	#include <iostream>
2	
3	using namespace std;
4	
5	int getIndex(char v, char vertices[], int n) {
6	for (int i = 0; i < n; i++) {
7	if (vertices[i] == v) return i;
8	}
9	return -1;

```

10 }
11
12 int main() {
13     int n;
14     cin >> n;
15
16     char vertices[15];
17     for (int i = 0; i < n; i++) {
18         cin >> vertices[i];
19     }
20
21     int m;
22     cin >> m;
23
24     int matrix[15][15] = {0};
25
26     for (int i = 0; i < m; i++) {
27         char u, v;
28         int w;
29         cin >> u >> v >> w;
30
31         matrix[getIndex(u, vertices, n)][getIndex(v,
vertices, n)] = w;
32     }
33
34     cout << "Adjacency Matrix:\n ";
35
36     for (int i = 0; i < n; i++) {
37         cout << vertices[i] << " ";
38     }
39     cout << "\n";
40     for (int i = 0; i < n; i++) {
41         cout << vertices[i];
42         for (int j = 0; j < n; j++) {
43             cout << " " << matrix[i][j];
44         }
45         cout << "\n";
46     }
47
48     return 0;
49 }

```

B. Output Program



```
PS C:\Users\Legion PC\Documents\coding\cpp\module-6-graph-and-tree-jyaritta> g++ .\probl.cpp
PS C:\Users\Legion PC\Documents\coding\cpp\module-6-graph-and-tree-jyaritta> .\a.exe
5
A B C D E
6
A B 4
A C 2
B D 7
C B 1
C E 6
D C 3
Adjacency Matrix:
A B C D E
A 0 4 2 0 0
B 0 0 0 7 0
C 0 1 0 0 6
D 0 0 3 0 0
E 0 0 0 0 0
```

Gambar 19. Screenshot Hasil Jawaban Soal 1 Modul 6

C. Pembahasan

Pada baris [1 sampai 2] `stack.cpp`, mengimpor file header "`stack.h`" untuk mengenali `struct Stack` dan mengimpor library `<stdexcept>` untuk memunculkan pesan error dengan `throw`.

Algoritma ini mengubah adjacency list yang menggunakan susunan daftar list menjadi adjacency matrix atau tabel dua dimensi berukuran $N \times N$. Inputnya ada N yaitu jumlah vertex, nama vertex, jumlah edge dan weight dari edge tersebut dan outputnya berupa graph representasi matriks.

Fungsi `getIndex` mencari indeks dari char vertex pada array vertices dengan menggunakan loop yang mengecek satu per satu elemen array. Jika nilai elemen sama dengan karakter yang dicari, kembalikan nilai indeksnya. Kalau tidak ketemu, return -1 untuk mengirimkan error. Time complexitynya adalah $O(N)$ karena melakukan iterasi sebanyak N , sedangkan space complexity adalah $O(1)$ karena tidak menggunakan memori atau fungsi tambahan.

Di fungsi main, menerima input nilai N dan nama-nama vertex ke array vertex. Lalu menerima input nilai M untuk jumlah edge untuk banyaknya sisi. Setelah itu inisialisasi array 2D matrix berukuran 15×15 yang berisikan 0. Lalu melakukan loop dari 0 sampai M untuk menambah edge ke graph.

Di dalam loop tersebut, program memanggil fungsi `getIndex` untuk mencari posisi baris dan kolom yang tepat, lalu memasukkan nilai weight edge ke dalam matrix dengan `matrix[getIndex(u, vertices, n)][getIndex(v, vertices, n)] = w;`. Terakhir, mencetak Adjacency Matrix, baris nama vertex dan print output graph melalui loop. Untuk time complexity-nya $O(M \times N + N^2)$ karena terdapat loop M untuk input edge dan di dalamnya memanggil fungsi `getIndex` dengan time complexity $O(N)$, ditambah nested loop untuk mencetak graph $O(N^2)$.

SOAL 2: Konversi Adjacency Matrix ke Adjacency List

Time Limit: 1.0 s
Memory Limit: 64 MB

Deskripsi Soal

Diberikan sebuah **directed weighted graph** yang direpresentasikan dalam bentuk **adjacency matrix** berukuran $N \times N$. Setiap vertex memiliki label berupa satu karakter unik.

Nilai matrix pada baris i dan kolom j menunjukkan bobot edge dari vertex ke- i menuju vertex ke- j . Jika nilainya 0, berarti tidak terdapat edge dari vertex ke- i menuju vertex ke- j .

Tugas Anda adalah mengubah adjacency matrix tersebut menjadi **adjacency list**. Untuk setiap vertex, cetak semua vertex tujuan yang dapat dicapai langsung beserta bobot edge-nya. Urutan vertex tujuan mengikuti urutan label pada input.

Format Input

Input diberikan dalam format berikut.

N

V1 V2 ... VN A11 A12 ... A1N A21 A22 ... A2N

...

AN1 AN2 ... ANN

Keterangan:

- N adalah jumlah vertex.

- V_i adalah label vertex ke- i .

- A_{ij} adalah nilai matrix pada baris ke- i dan kolom ke- j .

-Jika $A_{ij} > 0$, maka terdapat edge dari V_i menuju V_j dengan bobot A_{ij} .

-Jika $A_{ij} = 0$, maka tidak terdapat edge dari V_i menuju V_j .

Format Output

Cetak adjacency list untuk setiap vertex dengan format berikut.

Adjacency List:

V1 -> (X1,w1), (X2,w2), ...

V2 -> (X1,w1), (X2,w2), ...

...

VN -> ...

Jika sebuah vertex tidak memiliki edge keluar, cetak tanda - setelah simbol ->.

Constraints

$$-2 \leq N \leq 10$$

$$- 0 \leq A_{ij} \leq 1000$$

-Label vertex berupa karakter alfabet kapital A sampai Z.Semua label vertex dijamin unik.

- Nilai diagonal utama matrix dijamin 0.
- Nilai 0 berarti tidak ada edge.
- Nilai positif berarti bobot edge.
- Matrix merepresentasikan graph berarah dan berbobot.

Contoh Kasus

Input	Output
5 A B C D E 0 4 2 0 0 0 0 0 7 0 0 1 0 0 6 0 0 3 0 0 0 0 0 0 0	Adjacency List: A -> (B,4), (C,2) B -> (D,7) C -> (B,1), (E,6) D -> (C,3) E -> -

Penjelasan Contoh

Pada baris vertex A, nilai positif muncul pada kolom B dan C. Karena itu, adjacency list untuk A adalah (B,4), (C,2).

Pada baris vertex E, semua nilai bernilai 0, sehingga vertex E tidak memiliki edge keluar dan output-nya adalah E -> -.

A. Source Code

Tabel 22 Source code prob2.cpp Modul 6

```
1 #include <iostream>
2
3 using namespace std;
4
5 int main() {
6     int n;
7     cin >> n;
8
9     char vertices[15];
10    for (int i = 0; i < n; i++) {
11        cin >> vertices[i];
12    }
13
14    int matrix[15][15];
15    for (int i = 0; i < n; i++) {
16        for (int j = 0; j < n; j++) {
17            cin >> matrix[i][j];
18        }
19    }
20
21    cout << "Adjacency List:\n";
22
23    for (int i = 0; i < n; i++) {
24        cout << vertices[i] << " -> ";
25
26        bool isFirst = true;
27        for (int j = 0; j < n; j++) {
28            if (matrix[i][j] > 0) {
29                if (!isFirst) {
30                    cout << ", ";
31                }
32                cout << "(" << vertices[j] << ", " <<
matrix[i][j] << ")";
33                isFirst = false;
34            }
35        }
36        if (isFirst) {
37            cout << "-";
38        }
39        cout << "\n";
40    }
41
42    return 0;
43 }
```

B. Output Program

```
PS C:\Users\Legion PC\Documents\coding\cpp\module-6-graph-and-tree-jyaritta> g++ .\prob2.cpp
PS C:\Users\Legion PC\Documents\coding\cpp\module-6-graph-and-tree-jyaritta> .\a.exe
5
A B C D E
0 4 2 0 0
0 0 0 7 0
0 1 0 0 6
0 0 3 0 0
0 0 0 0 0
Adjacency List:
A -> (B,4), (C,2)
B -> (D,7)
C -> (B,1), (E,6)
D -> (C,3)
E -> -
```

Gambar 20. Screenshot Hasil Jawaban Soal 2 Modul 6

C. Pembahasan

Algoritma ini mengubah adjacency matrix menjadi adjacency list. Inputnya ada nilai N untuk jumlah vertex, daftar nama vertex, dan graph yang dalam bentuk adjacency matrix atau tabel berukuran $N \times N$.

Pada fungsi `main()` pertama-tama melakukan input nilai N dan membaca nama-nama vertex atau simpul ke dalam array `vertices`. Setelah itu, lakukan inisialisasi array matrix berukuran 15×15 dengan nested loop `i` dan `j` dari 0 sampai N untuk menaruh input nilai bobot tadi ke dalam array matrix.

Kemudian untuk mencetak output, program melakukan nested loop lagi. Loop luar `i` digunakan untuk iterasi vertex dengan `vertices[i]` diikuti dengan tanda `"->"`. Setelah itu loop berikutnya di dalam loop luar, terdapat variabel boolean `isFirst` bernilai `true` untuk mengatur penempatan tanda koma pada nilai pertama yang muncul. Permisalan, pertama program akan mengecek apakah `isFirst` aktif, karena nilai yang pertama tercetak maka koma tidak ikut tercetak. Setelah nilai pertama keluar maka nilai `isFirst` akan jadi `false`, setelah ini program akan mencetak koma. Jika loop `j` selesai dan `isFirst` masih bernilai `true` Dimana vertex awal tidak memiliki jalur sama sekali. Jadi, program hanya akan mencetak tanda `"-"`.

Time complexity untuk algoritma ini sendiri $O(N^2)$ karena bergantung pada nested loop yang matriks $N \times N$ dengan space complexity $O(1)$ karena ukuran matrix sudah ditetapkan di awal sehingga tidak berubah-ubah sesuai nilai input N .

Soal 3: BFS: Jarak Terpendek Keluar Labirin

Time Limit: 1.0 s
Memory Limit: 64 MB

Deskripsi Soal

Sebuah labirin direpresentasikan sebagai grid berukuran $R \times C$. Setiap sel pada grid memiliki nilai sebagai berikut.

- 0 menyatakan jalan yang dapat dilewati.
- 1 menyatakan dinding yang tidak dapat dilewati.

Diberikan posisi awal dan posisi tujuan. Anda harus menentukan jumlah langkah minimum dari posisi awal menuju posisi tujuan menggunakan algoritma **Breadth-First Search** (BFS).

Dari sebuah sel, Anda hanya boleh bergerak ke empat arah: atas, bawah, kiri, dan kanan. Pergerakan diagonal tidak diperbolehkan. Beberapa jalur pada labirin dapat berupa jalan buntu, sehingga algoritma harus tetap memproses kemungkinan jalur secara sistematis.

Format Input

Input diberikan dalam format berikut.

```
R C
G11 G12 ... G1C
G21 G22 ... G2C
...
GR1 GR2 ... GRC
SR SC
FR FC
```

Keterangan:

- R adalah jumlah baris grid.
- C adalah jumlah kolom grid.
- G_{ij} adalah nilai sel pada baris ke- i dan kolom ke- j .
- (S_R, S_C) adalah posisi awal.
- (F_R, F_C) adalah posisi tujuan.

Indeks baris dan kolom dimulai dari 0.

Format Output

Cetak satu bilangan bulat.

```
X
```

Keterangan:

- X adalah jumlah langkah minimum dari posisi awal ke posisi tujuan.
- Jika posisi tujuan tidak dapat dicapai, cetak -1.

Constraints

- $1 \leq R \leq 100$
- $1 \leq C \leq 100$
- G_{ij} hanya bernilai 0 atau 1.
- $0 \leq S_R < R$
- $0 \leq S_C < C$
- $0 \leq F_R < R$
- $0 \leq F_C < C$
- Sel start dan finish dijamin bernilai 0.
- Pergerakan hanya boleh dilakukan ke sel bernilai 0.
- Algoritma yang digunakan wajib BFS.

Contoh Kasus

Input	Output
<pre> 8 10 0 0 0 0 1 0 0 0 0 0 1 1 1 0 1 0 1 1 1 0 0 0 1 0 0 0 0 0 1 0 0 1 1 1 1 1 1 0 1 0 0 0 0 0 0 0 1 0 0 0 1 1 1 1 1 0 1 1 1 0 0 0 0 0 1 0 0 0 1 0 0 1 1 0 0 0 1 0 0 0 0 0 7 9 </pre>	16

Penjelasan Contoh

Posisi awal berada pada $(0, 0)$ dan posisi tujuan berada pada $(7, 9)$. Labirin memiliki beberapa percabangan dan jalan buntu, misalnya cabang menuju area kiri bawah dan cabang menuju sel $(2, 1)$.

Dengan BFS, setiap sel diproses berdasarkan jarak terdekat dari start. Jarak minimum dari $(0, 0)$ ke $(7, 9)$ adalah 16 langkah, sehingga output yang dicetak hanya 16.

A. Source Code

Tabel 23. Source code prob3.cpp Modul 6

1	#include <iostream>
2	
3	using namespace std;
4	
5	struct State {
6	int r, c, dist;
7	};
8	
9	State q[10001];
10	int head = 0;
11	int tail = 0;
12	
13	void push(int r, int c, int d) {
14	q[tail].r = r;

```

15     q[tail].c = c;
16     q[tail].dist = d;
17     tail++;
18 }
19
20 State pop() {
21     State s = q[head];
22     head++;
23     return s;
24 }
25
26 bool isEmpty() {
27     return head == tail;
28 }
29
30 int grid[101][101];
31 bool visited[101][101];
32
33 int dr[] = {-1, 1, 0, 0};
34 int dc[] = {0, 0, -1, 1};
35
36 int main() {
37     int R, C;
38     cin >> R >> C;
39
40     for (int i = 0; i < R; i++) {
41         for (int j = 0; j < C; j++) {
42             cin >> grid[i][j];
43             visited[i][j] = false;
44         }
45     }
46
47     int SR, SC, FR, FC;
48     cin >> SR >> SC;
49     cin >> FR >> FC;
50
51     push(SR, SC, 0);
52     visited[SR][SC] = true;
53
54     int ans = -1;
55
56     while (!isEmpty()) {
57         State curr = pop();
58
59         if (curr.r == FR && curr.c == FC) {
60             ans = curr.dist;
61             break;
62         }
63
64         for (int i = 0; i < 4; i++) {
65             int nr = curr.r + dr[i];
66             int nc = curr.c + dc[i];

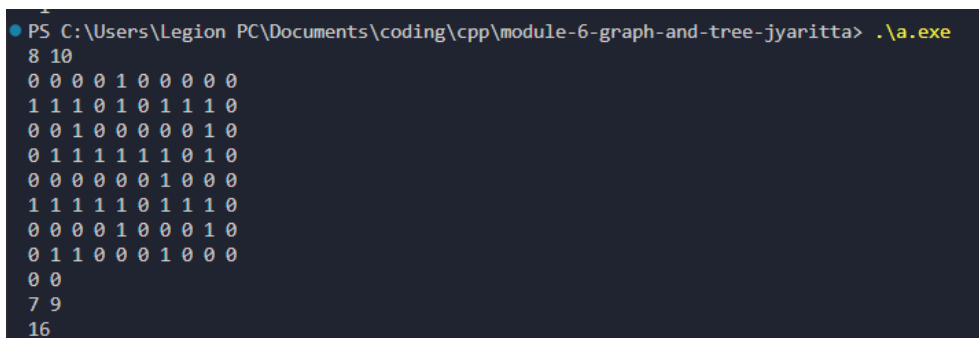
```

```

65
66         if (nr >= 0 && nr < R && nc >= 0 && nc < C) {
67             if (grid[nr][nc] == 0 && !visited[nr][nc]) {
68                 visited[nr][nc] = true;
69                 push(nr, nc, curr.dist + 1);
70             }
71         }
72     }
73 }
74 cout << ans << "\n";
75 return 0;
76 }
77
78

```

B. Output Program



```

PS C:\Users\Legion PC\Documents\coding\cpp\module-6-graph-and-tree-jyaritta> .\a.exe
8 10
0 0 0 0 1 0 0 0 0 0
1 1 1 0 1 0 1 1 1 0
0 0 1 0 0 0 0 0 1 0
0 1 1 1 1 1 1 0 1 0
0 0 0 0 0 0 1 0 0 0
1 1 1 1 1 0 1 1 1 0
0 0 0 0 1 0 0 0 1 0
0 1 1 0 0 0 1 0 0 0
0 0
7 9
16

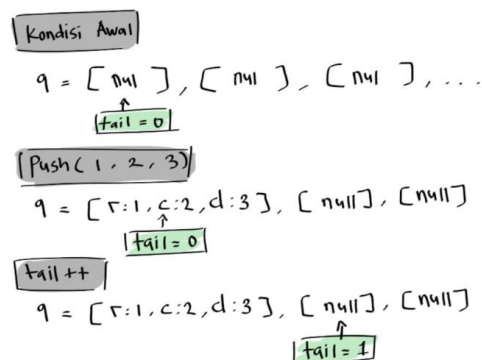
```

Gambar 21. Screenshot Hasil Jawaban Soal 3 Modul 6

C. Pembahasan

Algoritma berikut pencarian rute terpendek dalam sebuah labirin $R \times C$ menggunakan metode Breadth-First Search (BFS). Terdapat struct State yang berfungsi untuk menyimpan posisi baris r , kolom c , dan jumlah jarak $dist$. Selanjutnya terdapat deklarasi queue berupa array q berkapasitas 10001 elemen, bertambah 1 dari maksimal untuk jaga-jaga.

Terdapat fungsi `push` untuk memasukkan State baru ke posisi `tail` lalu menambah nilai `tail`, sebagai contoh pada gambar visualisasi di bawah:



Selanjutnya ada fungsi `pop` untuk mengambil `State` di posisi `head` lalu menambah nilai `head`, dan fungsi `isEmpty` untuk mengecek apakah queue kosong atau kondisi `head == tail`. Operasi pada queue manual ini memiliki time complexity $O(1)$.

Selanjutnya ada array `grid` untuk menyimpan bentuk labirin, array boolean `visited` sebagai penanda suatu petak sudah pernah dikunjungi dan array `dr` dan `dc` yang berisi perubahan koordinat atas, bawah, kiri, kanan.

Masuk ke fungsi `main()` dimulai dengan menginput `R`, `C`. Lalu, melakukan nested loop untuk membaca matriks labirin ke dalam array `grid` dan sekaligus menginisialisasi array `visited` dengan nilai `false`. Setelah itu, program menerima input koordinat `Start` dan `Finish` dimana `SR`, `SC` adalah `Start Row` dan `Start Column` posisi awal, `FR`, `FC` untuk `Finish Row`, `Finish Column` untuk posisi tujuan.

Proses BFS dimulai dengan memanggil `push` untuk memasukkan posisi `Start` ke antrean dengan nilai jarak 0, dan langsung menandai titik `Start` tersebut pada array `visited` menjadi `true`.

Program menginisialisasi variabel `ans` dengan nilai `-1` jika jalan sudah buntu. Selama `!isEmpty()`, program akan melakukan `pop State` di depan antrean sebagai titik `curr`. Jika titik ini sama dengan titik `Finish`, program mencatat jaraknya di `ans` dan langsung melakukan `break` untuk menghentikan pencarian, karena BFS jika sudah mencapai target adalah maka itu rute terpendek.

Untuk algoritma ini, time complexity adalah $O(R \times C)$ karena worst case dimana misalnya tidak ada dinding pada labirin, algoritma BFS akan mengunjungi setiap petak di dalam grid tepat satu kali, dan memproses pengecekan 4 arah dengan $O(1)$ per petaknya dan untuk Space complexity-nya adalah $O(R \times C)$ karena algoritma ini memerlukan alokasi memori sebanyak jumlah keseluruhan petak di dalam labirin.

Soal 4: DFS: Menghitung Semua Jalur Keluar Labirin

Time Limit: 2.0 s
Memory Limit: 64 MB

Deskripsi Soal

Diberikan sebuah labirin berukuran $R \times C$ dengan aturan sel yang sama seperti soal sebelumnya. Anda harus menghitung banyaknya jalur sederhana dari posisi awal menuju posisi tujuan menggunakan algoritma **Depth-First Search** (DFS) rekursif.

Sebuah jalur disebut sederhana jika tidak mengunjungi sel yang sama lebih dari satu kali dalam jalur yang sama. Dari sebuah sel, Anda hanya boleh bergerak ke empat arah: atas, bawah, kiri, dan kanan.

Labirin dapat memiliki beberapa percabangan dan jalan buntu. Karena itu, DFS harus melakukan proses **backtracking** agar dapat mencoba semua kemungkinan jalur yang valid.

Format Input

Input diberikan dalam format berikut.

```
R C

G11 G12 ... G1C
G21 G22 ... G2C

...

GR1 GR2 ... GRC
SR SC

FR FC
```

Keterangan:

- R adalah jumlah baris grid.
- C adalah jumlah kolom grid.
- G_{ij} adalah nilai sel pada baris ke- i dan kolom ke- j .
- (S_R, S_C) adalah posisi awal.
- (F_R, F_C) adalah posisi tujuan.

Indeks baris dan kolom dimulai dari 0.

Format Output

Cetak satu bilangan bulat.

```
T
```

Keterangan:

- T adalah jumlah jalur sederhana dari posisi awal ke posisi tujuan.
- Jika tidak ada jalur yang dapat dilalui, cetak 0.

Constraints

- $1 \leq R \leq 7$

- $1 \leq C \leq 7$
- G_{ij} hanya bernilai 0 atau 1.
- $0 \leq S_R < R$
- $0 \leq S_C < C$
- $0 \leq F_R < R$
- $0 \leq F_C < C$
- Sel start dan finish dijamin bernilai 0.
- Pergerakan hanya boleh dilakukan ke sel bernilai 0.
- Satu jalur tidak boleh mengunjungi sel yang sama lebih dari satu kali.
- Jumlah jalur valid pada test case dijamin tidak lebih dari 100000.
- Algoritma yang digunakan wajib DFS rekursif dengan backtracking.

Contoh Kasus

Input	Output
<pre> 5 6 0 0 0 1 0 0 0 1 0 0 0 1 0 1 0 1 0 0 0 0 0 1 1 0 1 1 0 0 0 0 0 0 4 5 </pre>	4

Penjelasan Contoh

Posisi awal berada pada $(0, 0)$ dan posisi tujuan berada pada $(4, 5)$. Labirin memiliki lebih dari satu jalur valid serta beberapa cabang yang tidak menghasilkan solusi, misalnya cabang menuju sel $(0, 5)$.

Dengan DFS dan backtracking, seluruh jalur sederhana dari start ke finish dapat dihitung. Pada contoh ini terdapat 4 jalur valid, sehingga output yang dicetak hanya 4.

A. Source Code

Tabel 24. Source code prob4.cpp Modul 6

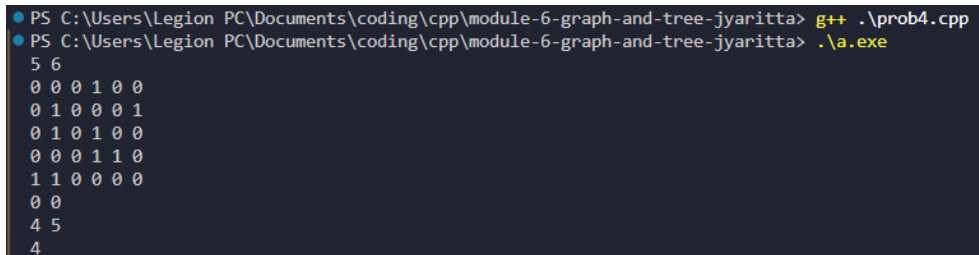
```
1  #include <iostream>
2
3  using namespace std;
4
5  struct Labirin {
6      int grid[10][10];
7      bool visited[10][10];
8      int R, C;
9      int FR, FC;
10     int total_jalur;
11 };
12
13 void dfs(Labirin& map, int r, int c) {
14     if (r == map.FR && c == map.FC) {
15         map.total_jalur++;
16         return;
17     }
18
19     int dr[] = {-1, 1, 0, 0};
20     int dc[] = {0, 0, -1, 1};
21
22     for (int i = 0; i < 4; i++) {
23         int nr = r + dr[i];
24         int nc = c + dc[i];
25
26         if (nr >= 0 && nr < map.R && nc >= 0 && nc <
map.C) {
27             if (map.grid[nr][nc] == 0 &&
!map.visited[nr][nc]) {
28                 map.visited[nr][nc] = true;
29                 dfs(map, nr, nc);
30                 map.visited[nr][nc] = false;
31             }
32         }
33     }
34 }
35
36 int main() {
37     Labirin map;
38     map.total_jalur = 0;
39     cin >> map.R >> map.C;
40
41     for (int i = 0; i < map.R; i++) {
42         for (int j = 0; j < map.C; j++) {
43             cin >> map.grid[i][j];
44             map.visited[i][j] = false;
45         }
46     }
47 }
```

```

48     int SR, SC;
49     cin >> SR >> SC;
50     cin >> map.FR >> map.FC;
51
52     map.visited[SR][SC] = true;
53     dfs(map, SR, SC);
54     cout << map.total_jalur << "\n";
55     return 0;
56 }

```

B. Output Program



```

PS C:\Users\Legion PC\Documents\coding\cpp\module-6-graph-and-tree-jyaritta> g++ .\prob4.cpp
PS C:\Users\Legion PC\Documents\coding\cpp\module-6-graph-and-tree-jyaritta> .\a.exe
5 6
0 0 0 1 0 0
0 1 0 0 0 1
0 1 0 1 0 0
0 0 0 1 1 0
1 1 0 0 0 0
0 0
4 5
4

```

Gambar 22. Screenshot Hasil Jawaban Soal 4 Modul 6

C. Pembahasan

Algoritma berikut menggunakan metode Depth-First Search dengan teknik Backtracking. Cara kerja algoritma ini ia akan memilih satu jalur dan menelusurinya hingga sampai ke tujuan atau setidaknya jalan buntu, lalu mundur untuk mencoba jalur yang lain.

Pertama membuat `struct Labirin` untuk menaruh data labirinnya di sana. Selanjutnya ada fungsi `dfs` yang menerima parameter dari `struct Labirin` beserta posisi baris `r` dan kolom `c` saat ini. Fungsi DFS ini bertugas mengecek apakah sudah sampai di tempat tujuan, melihat ke 4 arah untuk memastikan jalan berikutnya bukan tembok atau tidak keluar batas labirin, sebelum mulai melangkah.

Di dalam fungsi `main()`, pertama program akan menyiapkan peta `Labirin`, mendeklarasikan total jalur yang ada lalu membaca ukuran baris dan kolom untuk `Labirin`. Selanjutnya ada perulangan untuk membaca input angka 0 dan 1 ke dalam `grid`. Lalu program akan melakukan perulangan untuk membaca input angka dan memasukkannya ke dalam `grid`. Selanjutnya program akan meminta input dimana tempat start dan finish. Sebelum jalan, melakukan tempat awal menjadi `visited = true` agar tidak terjadi kesalahan kembali lagi ke garis start. Setelah itu, panggil fungsi `dfs` untuk mencari total jalur.

Time complexity algoritma ini adalah $O(4^{R \times C})$ karena dalam kondisi worst case dimana misalnya labirin tanpa dinding sel dapat bercabang hingga ke 4 arah yang berbeda yang menghasilkan banyak jalur yang bertambah secara eksponensial dan space complexity-nya adalah $O(R \times C)$ karena memori hanya digunakan untuk menyimpan array 2D `grid` dan `visited` di dalam `struct` dan lainnya.

Soal 5: Tree Traversal

Time Limit: 1.0 s
Memory Limit: 64 MB

Deskripsi Soal

Diberikan N bilangan bulat. Masukkan seluruh bilangan tersebut ke dalam sebuah **Red-Black Tree** secara berurutan sesuai input. Jika terdapat bilangan duplikat, bilangan tersebut diabaikan dan tidak dimasukkan kembali ke tree.

Setelah RBTree terbentuk, diberikan Q query. Setiap query meminta salah satu hasil traversal berikut.

- `PREORDER`, yaitu urutan root, subtree kiri, lalu subtree kanan.
- `INORDER`, yaitu urutan subtree kiri, root, lalu subtree kanan.
- `POSTORDER`, yaitu urutan subtree kiri, subtree kanan, lalu root.
- `ALL`, yaitu mencetak ketiga traversal sekaligus.

Tugas Anda adalah mencetak hasil traversal sesuai query yang diberikan.

Format Input

Input diberikan dalam format berikut.

```
N
A1 A2 ... AN
Q
T1
T2
...
TQ
```

Keterangan:

- N adalah banyaknya bilangan yang akan dimasukkan ke RBTree.
- A_i adalah bilangan ke- i .
- Q adalah banyaknya query traversal.
- T_i adalah jenis traversal yang diminta.

Format Output

Untuk setiap query, cetak hasil traversal sesuai format berikut.

```
[Preorder] : daftar_angka
[Inorder]   : daftar_angka
[Postorder] : daftar_angka
```

Jika query adalah `ALL`, cetak ketiga baris traversal dengan urutan Preorder, Inorder, lalu Postorder.

Jika setelah penghapusan duplikat tree tetap kosong, cetak:

Tree kosong. Tidak ada yang bisa ditampilkan.

Constraints

- $0 \leq N \leq 1000$
- $-1000000000 \leq A_i \leq 1000000000$
- $1 \leq Q \leq 20$
- T_i hanya salah satu dari `PREORDER`, `INORDER`, `POSTORDER`, atau `ALL`.

Contoh Kasus

Input	Output
7	[Preorder] : 50 30 20 40 70 60 80
50 30 70 20 40 60 80	[Inorder] : 20 30 40 50 60 70 80
4	[Postorder] : 20 40 30 60 80 70 50
PREORDE	[Preorder] : 50 30 20 40 70 60 80
R	[Inorder] : 20 30 40 50 60 70 80
INORDER	[Postorder] : 20 40 30 60 80 70 50
POSTORD	
ER ALL	

Penjelasan Contoh

Bilangan dimasukkan ke RBTREE secara berurutan. Nilai 50 menjadi root. Nilai yang lebih kecil dari 50 masuk ke subtree kiri, sedangkan nilai yang lebih besar masuk ke subtree kanan.

Hasil `INORDER` pada RBTREE selalu menghasilkan urutan nilai dari kecil ke besar. Pada contoh ini, hasil inorder adalah 20 30 40 50 60 70 80, sehingga struktur RBTREE yang dibentuk sudah konsisten dengan aturan RBTREE.

A. Source Code

Tabel 25. Source code prob5.cpp Modul 6

```
1  #include <iostream>
2  #include "RedBlackTree.h"
3
4  using namespace std;
5
6  bool isMatch(const char* a, const char* b) {
7      int i = 0;
8      while (a[i] != '\0' && b[i] != '\0') {
9          if (a[i] != b[i]) return false;
10         i++;
11     }
12     return a[i] == '\0' && b[i] == '\0';
13 }
14
15 void Preorder(const RedBlackTree::Node* node, const
RedBlackTree::Node* nilNode, bool& isFirst) {
16     if (node == nilNode || node->isNil) return;
17
18     if (!isFirst) cout << " ";
19     cout << node->key;
20     isFirst = false;
21
22     Preorder(node->left, nilNode, isFirst);
23     Preorder(node->right, nilNode, isFirst);
24 }
25
26 void Inorder(const RedBlackTree::Node* node, const
RedBlackTree::Node* nilNode, bool& isFirst) {
27     if (node == nilNode || node->isNil) return;
28
29     Inorder(node->left, nilNode, isFirst);
30
31     if (!isFirst) cout << " ";
32     cout << node->key;
33     isFirst = false;
34
35     Inorder(node->right, nilNode, isFirst);
36 }
37
38 void Postorder(const RedBlackTree::Node* node, const
RedBlackTree::Node* nilNode, bool& isFirst) {
39     if (node == nilNode || node->isNil) return;
40
41     Postorder(node->left, nilNode, isFirst);
```

```

42     Postorder(node->right, nilNode, isFirst);
43
44     if (!isFirst) cout << " ";
45     cout << node->key;
46     isFirst = false;
47 }
48
49 int main() {
50     int n;
51     cin >> n;
52
53     RedBlackTree rbt;
54
55     for (int i = 0; i < n; i++) {
56         int val;
57         cin >> val;
58         if (!rbt.contains(val)) {
59             rbt.insert(val);
60         }
61     }
62
63     int q;
64     cin >> q;
65
66     if (rbt.empty()) {
67         cout << "Tree kosong\n";
68         return 0;
69     }
70
71     for (int i = 0; i < q; i++) {
72         char query[20];
73         cin >> query;
74
75         if (isMatch(query, "PREORDER") || isMatch(query,
76 "ALL")) {
77             cout << "[Preorder] : ";
78             bool isFirst = true;
79             Preorder(rbt.root(), rbt.nil(), isFirst);
80             cout << "\n";
81         }
82         if (isMatch(query, "INORDER") || isMatch(query,
83 "ALL")) {
84             cout << "[Inorder] : ";
85             bool isFirst = true;
86             Inorder(rbt.root(), rbt.nil(), isFirst);
87             cout << "\n";
88         }
89     }
90 }

```

```

87         if (isMatch(query, "POSTORDER") || isMatch(query,
88 "ALL")) {
89             cout << "[Postorder]: ";
90             bool isFirst = true;
91             Postorder(rbt.root(), rbt.nil(), isFirst);
92             cout << "\n";
93         }
94     }
95     return 0;
96 }
97

```

B. Output Program

```

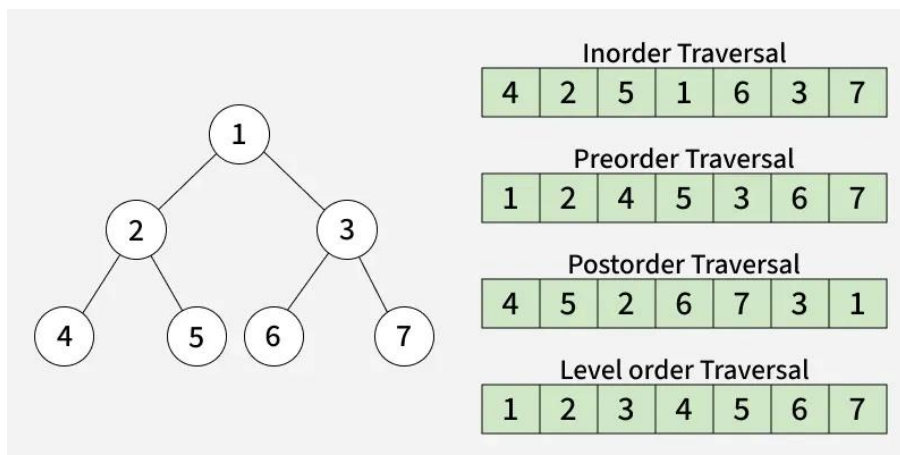
PS C:\Users\Legion PC\Documents\coding\cpp\module-6-graph-and-tree-jyaritta> g++ .\prob5.cpp .\RedBlackTree.cpp -o prob5
PS C:\Users\Legion PC\Documents\coding\cpp\module-6-graph-and-tree-jyaritta> .\prob5.exe
7
50 30 70 20 40 60 80
4
PREORDER
INORDER
POSTORDER
ALL
[Preorder] : 50 30 20 40 70 60 80
[Inorder] : 20 30 40 50 60 70 80
[Postorder]: 20 40 30 60 80 70 50
[Preorder] : 50 30 20 40 70 60 80
[Inorder] : 20 30 40 50 60 70 80
[Postorder]: 20 40 30 60 80 70 50

```

Gambar 23. Screenshot Hasil Jawaban Soal 5 Modul 6

C. Pembahasan

Kode prob5.cpp merupakan implementasi algoritma struktur data Red-Black Tree berupa Preorder, Inorder, dan Postorder berdasarkan query yang diminta. Singkatnya, penjelasan sederhananya ada di gambar berikut:



Fungsi `isMatch`, digunakan untuk membandingkan dua string tanpa menggunakan library `<string>` dengan melakukan iterasi `while` untuk mengecek karakter satu per satu. Jika ada yang berbeda, kembalikan `false`, ini dipakai untuk mengecek query user seperti "PREORDER", "INORDER", "POSTORDER".

Fungsi `Preorder`, menelusuri tree dari root ke kiri lalu ke kanan dengan membuat base case jika node sekarang adalah `nilNode` atau `isNil` yakni kosong, maka `return`. Kalau bukan, cetak nilai `node->key`. Terdapat juga pengecekan boolean `isFirst` untuk memastikan spasi saat pencetakan nanti. Lalu rekursif ke kiri, dan rekursif lagi ke kanan.

Fungsi `Inorder`, menelusuri tree dari kiri ke root lalu ke kanan, cara kerjanya mirip dengan `preorder`, tetapi urutan eksekusinya adalah rekursif ke kiri dahulu, lalu cetak nilai `node->key`, baru rekursif ke kanan.

Fungsi `Postorder`, menelusuri tree dari kiri ke kanan lalu ke root dengan rekursif ke kiri terlebih dahulu, lalu rekursif ke kanan, dan diakhiri dengan mencetak nilai `node->key`.

Ketiga fungsi traversal ini memiliki time complexity $O(N)$ karena setiap node di dalam tree dikunjungi satu kali dan space complexity $O(\log N)$ yang berasal dari call stack memori untuk fungsi rekursi.

Selanjutnya pada fungsi `main()`, pertama-tama melakukan `inout` untuk `n` dan mendeklarasikan `RedBlackTree` sebagai `rbt`. Selanjutnya program akan melakukan perulangan `n` kali untuk membaca angka dan setiap angka dicek menggunakan `rbt.contains(val)` agar bilangan duplikat diabaikan. Lalu nilai tersebut di-insert ke dalam tree. Selanjutnya program menerima input `q` untuk jumlah query. Jika tree kosong program mencetak pesan "Tree kosong" dan `return 0` untuk berhenti. Jika tidak kosong, program melakukan iterasi sebanyak `q` kali untuk membaca string query dan menggunakan perulangan menggunakan fungsi `isMatch`, dengan mencocokkan input dengan "PREORDER", "INORDER", "POSTORDER", atau "ALL". Jika cocok, setiap kondisi akan memanggil fungsi traversal yang tepat dengan `isMatch`.

Time complexity keseluruhan program ini adalah $O(N \log N + Q \times N)$ karena memasukkan `N` elemen ke dalam RBT memakan sebesar $O(\log N)$ per elemennya, dan setiap eksekusi dari `Q` query traversal $O(N)$. lalu, space complexity nya adalah $O(N)$ karena memerlukan alokasi memori untuk menyimpan `N` elemen di dalam node `RedBlackTree`.

TAUTAN GIT

Berikut adalah tautan untuk semua source code yang telah dibuat:

<https://github.com/jyaritta/Algoritma-Struktur-Data>